Defect register

Project: network-visualiser
Started: 22 September 2026, at the close of the post-Gate-D documentation batch
Status: living document

How this register works

- Every defect gets an entry when it is logged, with a number, the date, how it was found, and a priority.
- An entry is closed by a commit, and the entry cites that commit. It is never deleted: only its status changes.
- A disposition that decides *not* to fix something is still a disposition, and the reason is written down.
- Find everything about one defect with `git --no-pager log --oneline --grep "D-<n>"`.

Coverage. This register starts with the documentation batch. Defects D-1 to D-41 that are not listed below were dispositioned before it, in the gate records (`docs/gate_a_record.md` to `docs/gate_d_record.md`), in `docs/day17_gate_c_addendum.md`, and in their commit messages. They are not restated here from memory.

Open

ID	Logged	Defect	Priority
D-48	22 Sep	3-D layout was never swept separately	Low
D-53	23 Sep	Two separate `coverage_counts` implementations: `assistant.py` L480 and `summaries.py` L240	Medium
D-54	23 Sep	Silent fall-throughs on an unrecognised display state: `canvas._point_colour`, `card.group_label`, `centrality` `is_uncertain.fillna(False)`	Medium
D-56	24 Sep	Plotly's 3D renderer draws marker.symbol "circle-open" as nothing; the fourth display state was invisible on the canvas while present and hoverable in the figure	High
D-60	28 Sep	`state.get_corrections()` takes no `store` parameter, so it has no injection seam	Low
D-69	29 Sep	Every classification figure in the project is a point estimate from a non-deterministic process; the spread was never measured	Medium

ID	Logged	Defect	Priority
D-72	6 Oct	A keyed build blocks for minutes behind a spinner; a disconnect does not stop the spending, and a restart loses the work	Medium
D-73	6 Oct	`_partition_cached` checks the cache but not the current batch, so titles sharing a normalised key are sent separately, paid for separately, and can disagree within one run	Medium
D-86	10 Oct	`session_io.STALE_RESULT_NOTE` still says “The comparison below…”, a wording D-84 withdrew; the constant is never rendered	Low
D-87	10 Oct	One full-suite run raised an ERROR on a pure test (`test_welcome.py::test_a_non_integer_count_drops_the_size`) not reproduced in nine further runs; error text not captured	Medium
D-88	10 Oct	`docs/p2_9b_spike_record.md` says the encoder is shared across sessions (`st.cache_resource`); `embeddings._load_model()` builds a new one on every call, 1.32 s measured	Low
D-89	10 Oct	A refused session restore shows no message: `st.rerun()` after `_apply_upload` discards its `st.error`, and an earlier restore’s success report stays beside the refused file	Medium
D-90	10 Oct	An upload is built again without being uploaded again: “Use the demo instead” deletes the visitor’s corrections, notes and chat and rebuilds the same file; a rerun during a build throws the finished build away	High

ID	Logged	Defect	Priority
D-91	10 Oct	Step 6's tripwire fired: "Examining the path of torch.classes" appeared twice on a container provisioned fresh on the current config, so step 6's explanation for the leftover lines is wrong	Low

D-47 — Phase 1.5 evaluation may carry padded abstentions (closed 22 Sep)

Found: while writing the D-42 correction note for the methodology document. `src/bakeoff.py` classifies through `classify_claude` with the cache on, the same cache that held 118 padded records before D-42 was fixed.

Possible effect: a padded record removes an answer. That lowers coverage, and on a title whose true label is "no occupation" it counts as a correct abstention. Precision when the classifier commits is affected only through which titles remained answered. The locked decision not to re-tune config E rests on precision, so it is probably safe, but that has not been tested.

Next step: re-run the evaluation against the clean cache and compare coverage, correct abstentions and precision with the recorded figures. Costs API calls only for evaluation titles not already cached.

Result: `tools/remeasure_config_e.py`, offline. All 100 blind titles were cached and none was cached as unparsed; none changed prediction between the bake-off report (written 18 July) and the clean cache. Config E is unchanged at 63.5% precision, 74.0% coverage, 0 fabrications. The decision rule set before the run (precision at or above the 60% human ceiling, and 0 fabrications) is met, so the config E decision stands. Whether the cache was ever reset since July is not remembered, but a reset would have re-sampled the model, and zero changes across 100 titles makes that implausible. Dated note added to the methodology document.

D-48 — 3-D layout was never swept separately

Found: during D-29. The UMAP sweep in `data/cache/umap_sweep_results.csv` is 2-D only. The 3-D layout, which is what the app shows, reuses the 2-D values.

Why low: the values were chosen for product reasons (clickable node spacing, and a metric known to favour small `n_neighbors`), not by maximising the metric, and those reasons carry over to 3-D. The sweep is offline and free if wanted.

D-49 — Dead HDBSCAN constants and a stale docstring in `config.py` (closed 22 Sep)

Found: during D-29. `HDBSCAN_MIN_CLUSTER_SIZE` and `HDBSCAN_MIN_SAMPLES` are read by nothing; HDBSCAN is permanently ruled out. The module docstring still mentions a data-source toggle, retired on Day 19.

Next step: delete the two constants and correct the docstring. A test run confirms nothing read them.

D-50 — Correction-miss container and mixed-kind restore render never executed (closed 22 Sep)

Found: Gate C step 2, logged then as a known untested path. At 442 rows a restore reports no misses, because the saved session was created against the current export and no title has moved inside it. The code path will otherwise first execute on a real user's second upload.

Recipe (no rebuild, no API spend):

1. Save a session from the app that holds at least one correction and one note.
2. Copy the saved JSON. In the copy only, change the `role` of one correction entry to a title that person does not hold.
3. Restore the copy.
4. Expected: the restore report shows the correction miss in its own container, naming the person and the reason, with corrections and notes counted separately and nothing merged or silently dropped.

Delete the edited copy afterwards; it holds real names.

Result: run manually on 22 September. The report read *Re-applied 3 of 4 corrections and 2 of 2 notes*, with the miss in its own container naming the person and the reason (*their title has changed*), a Dismiss control, and no crash. Every criterion passed. No code change.

D-52 — The deployed environment ran pyarrow 24.0.0, not the locked 25.0.0 (closed 22 Sep)

Found: by the P2.9b resource spike (`docs/p2_9b_spike_record.md`). At deploy, Community Cloud replaced the locked `pyarrow==25.0.0` with 24.0.0, citing a known segfault (apache/arrow#50471). The lock therefore did not describe what ran.

Fix: `pyarrow==24.0.0` pinned in `requirements.in`, with the reason. Both locks recompiled: only pyarrow changed in the runtime lock, and the dev lock did not change. Confirmed with `tools.verify_manifests`.

D-57 — A sentinel string replaced `None`, and every truthiness check inverted

Found: during the `KEY_USER` / `KEY_PROJECT` / `KEY_NONE` vocabulary rename, before the commit landed. The bug existed only in uncommitted work and never reached main. `src/dashboard/upload.py` read `has_key = bool(key_source(store))`. Under the old contract that was correct: `key_source()` returned `None` when no key was configured. Under the new one it always returns a non-empty string, so `has_key` would have been `True` for every visitor, including those with no key at all.

Effect had it shipped: every upload would have believed a key was present and attempted to classify with none configured. The keyless path — the one that makes the tool usable without a key — would have been unreachable, failing instead somewhere inside classification.

Why no test caught it: no test asserted either the old or the new return values, so the rename broke nothing visible. Truthiness is the problem: a call site that never names the value cannot be found by grepping for the value. The sweep above is the only way to find them, and it has to be run by hand at the moment the contract changes.

The general case, for next time: replacing `None` with a sentinel value is not a rename. It is a contract change that inverts every boolean use of the function, and those uses are invisible to a search for either the old value or the new one. Sweep every call site by function name before the change lands.

Fixed: `upload.py` compares explicitly against `KEY_NONE`. `key_source()` states in its docstring that its return is always truthy and must never be tested for truth. The docstring and its guard test landed in `dd51c29`; the call-site fix landed in the commit that made the upload panel key-aware. Both carry `D-57`, so `--grep` finds the pair.

D-58 — The demo count is a literal on the first screen (closed 29 Sep)

Found: during the P2.9b commit batch, in `sweep_count_surfaces` output. The figure is correct against `data/demo/network_nodes.csv` today.

Why it is logged anyway: the literal carries no provenance. Regenerating the synthetic export at another size leaves the welcome screen stating the old figure, with nothing failing — the sweep matches `N of M` constructions and this is a bare number in prose. And 442 is also the real network's size, because the demo was generated to match it, so a reader cannot tell from the code which network the number describes.

Next step: read the count from the demo table at render time, as `upload.py:114` does for a built network.

Fixed: `welcome.py` reads the count from `data/demo/provenance.json`, the same file the provenance line reads, through `load_demo_provenance()`. The count is omitted rather than guessed when the file cannot be read. Closed by the commit carrying D-58.

Why not the demo table, as planned: the welcome screen and the provenance line now read one record. The table stays the source for the dashboard header, so `tests/test_demo.py` asserts the recorded `people` equals the table's row count. `build_demo.py` writes both in one run, and that test is what keeps them one figure.

D-60 — `state.py`'s `get_corrections()` has no injectable store

Found: while scoping D-59's exempt list. A related but distinct defect, not the same shape: D-59 was three copies of one generic accessor; `get_corrections()` is the only owner of `CORRECTIONS_KEY` and reaches `st.session_state` directly, with no `store` parameter to inject a plain dict in its place.

Effect: `load_display_table()` has to reach into session state through a function it cannot inject into, and a test of corrections has to monkeypatch `st.session_state` rather than pass a dict — the same friction D-59 removed from `session_build.py`, `welcome.py` and `keys.py`.

Why low: correct today; the cost is in testability and being the one remaining un-injectable seam in the display-table assembly chain. Fix when next in that file, not on its own.

D-62 — Tests that read the environment pass according to the developer's shell

Found: twice during P2.9b, both times by the shell changing rather than by the suite. `test_the_panel_builds_without_a_cl` and `test_the_panel_holds_the_build_and_reports_it` each asserted on rendering that depends on whether a key is configured, without pinning it.

Swept, 28 Sep: the full suite run twice, once with `ANTHROPIC_API_KEY` unset and once with a fake value. 537 passed both times, so no further test depends on the ambient environment. Both known cases are fixed with `monkeypatch.delenv`.

Recipe, because grep cannot find these. A test can be key-dependent without naming any key symbol — both of these were. The only reliable check is to run the suite in both states:

```
$saved = $env:ANTHROPIC_API_KEY
$env:ANTHROPIC_API_KEY = $null; python -m pytest -q
$env:ANTHROPIC_API_KEY = "sk-ant-notarealkey"; python -m pytest -q
$env:ANTHROPIC_API_KEY = $saved
```

Next step: run this before P2.9. CI has a different environment again, and a deploy is the third state neither of these runs covers.

Priority: closed as swept; the recipe is the standing item.

D-63 — A default parameter recorded whose key paid, for every caller that did not say (closed 28 Sep)

Found: while wiring `own_key` into `render_notice`. `build_network` took `source: str = KEY_PROJECT`, and `upload.py`, its only dashboard caller, never passed `source`. Every build from the app recorded `"project"` in its provenance, including a build classified on the visitor's own pasted key. Reachable from `df38fa5`, when a pasted key could first drive a build.

Effect: a key that died mid-build on the visitor's own account read as *the classification service was unavailable*. A retry then wrote `source=key_source()`, and the same failure on the same screen read as *your API key was rejected*. So one failure had two contradictory explanations, depending on whether a retry had run. The profile card never received a source at all, so it always gave the project wording. `unclassified.py` keeps the two phrases apart precisely because each is a false claim in the other's case.

Why nothing failed: the one test on provenance asserted that `source` was truthy, and the default guaranteed that it was.

The general case: a default that asserts a specific, checkable fact nobody supplied. Same family as D-57: a value standing in for "not known" that reads as a known answer. `retry_unclassified` had always defaulted `source` to `None`; the two now agree.

Fixed: - `source` defaults to `None`, and provenance records it as absent. - `upload.py` passes `key_source(store)`, which is `KEY_NONE` on a keyless build. - `reason_phrase`, `pending_clause`, `render_notice` and `card.provenance_label` take `source` rather than an `own_key` bool. - `None` gets a third phrase, *the classification did not complete*, which claims nothing about whose key it was. - `app.py` passes the build's own recorded source to both the notice and the card, never the session's current key. - The regression guard in `tests/test_app.py` builds on the project key, has that key fail, pastes a user key, and asserts the notice still says *unavailable*.

D-64 — A classifying test without an injected cache writes to the real cache (closed 28 Sep)

Found: 28 Sep, in `git status`. A fabricated `"title 6"` answer (the fake client's `soc: 13, why: ok`) had landed in `data/cache/claude_classification.json`. It came from a D-63 test that ran a classifying build with `cache_path` unset. It was removed by hand and the test now uses `tmp_path`.

Why medium: this is D-42's failure mode by a different route: a wrong answer entering the permanent cache with nothing failing. A cached answer is served to every later build without a call, so a real user with a job title a test happened to use would get the fake classification. The only signal was a dirty `git status` among a dozen other modified files.

Next step: a session-scoped autouse fixture in `tests/conftest.py`. It points the cache at a temporary directory for the whole suite, so a test that forgets cannot reach the real file; a test that needs the real cache opts in explicitly. That makes the safe path the default, the same reasoning as the single loader. Its own commit.

Trap for whoever writes the fixture. Patching `src.claude_classifier.CACHE_PATH` alone redirects nothing: - `count_uncached_titles`, `classify_claude_outcome` and `classify_claude` bind `CACHE_PATH` as a default argument, evaluated once at import. - `build.py` imports its own copy by value.

The fixture would pass review and guard nothing. Resolve the default at call time (`cache_path=None`, meaning the module's `CACHE_PATH`) so there is one name to patch. Then have the fixture fail the session if the real file's hash changes, so the guard is checked rather than trusted. `embeddings.DEFAULT_CACHE_PATH` has the same shape.

Fixed: - `claude_classifier._resolve_cache()` looks `CACHE_PATH` up at call time. The three functions default `cache_path` to `None` and resolve through it. - `build.py` no longer imports its own copy; it passes `cache_path` through. - In `tests/conftest.py`, an autouse fixture redirects `CACHE_PATH` to `tmp_path` for every test. It is per-test rather than session-scoped, so tests cannot see each other's cached answers either. A test that needs the real cache must undo the patch deliberately. - A session-scoped backstop hashes the real file before and after the run and fails if it changed. - `tests/test_cache_isolation.py` calls each defaulting path with no cache path and asserts the answer landed in the redirected file. One of those calls is the exact build that wrote `"title 6"`.

Not covered: `embeddings.DEFAULT_CACHE_PATH`. The build passes `embeddings_cache=None` and never reaches it, but a test calling `compute_embeddings` directly without a path would write `data/cache/embeddings.npy`. That file is a cache of vectors, not answers, so it cannot serve a wrong classification. Left until something calls it.

D-65 — Corrections and notes outlived the network they were made on (closed 29 Sep)

Found: 29 Sep, while scoping session expiry. Expiry had to clear corrections with the build, because a correction is `{person_index: code}` and the next upload would inherit it. Checking where that already

happens found that nothing did: `upload.py` replaced the build with `set_built`, and neither Start over nor "Use the demo instead" touched corrections or notes. `clear_all_corrections()` had no callers.

Effect: the welcome screen's two options, taken in order. A visitor explores the demo and corrects person 87, then uploads their own export. Their person 87 is shown with a group they never gave, provenance *corrected by you* — a false claim about a real individual, on the path the first screen invites. Notes, chat history, group summaries, skips and the selection carried over the same way.

Why no test caught it: every test of the upload panel started from an empty store. None put work on screen before changing the network.

Priority: High. It is the D-42 failure (a claim nobody made, stated as fact) on the most ordinary sequence in the app.

Fixed: `session_build.forget_network_work()` drops everything keyed to the network on screen. `clear_built()` calls it, so Start over and "Use the demo instead" get it; the upload panel calls `clear_built()` before `set_built()`. The pending retry offer, previously cleared separately there, is one of the keys. It keeps the API key, the chat allowance (resetting it would make "upload again" a way round the free tier), the self-enrichment state, and the restore file marker. A retry calls `set_built()` alone and keeps the work: same network. `tests/test_session_build.py` covers each path, including the retry.

D-66 — The dashboard raised on every run: an expander inside an expander (closed 29 Sep)

Found: 29 Sep, driving `app.py` through Streamlit's AppTest to check session expiry. With the demo and with a built network, every run ended in `StreamlitAPIException: Expanders may not be nested inside other expanders`. The welcome screen was unaffected.

Cause: `df38fa5` (P2.9b slice 2) put the key field in an `st.expander` inside `upload._key_section`, so it would read as optional on the welcome screen. The sidebar calls `render_upload_panel` from inside its own "Your network" expander, so there the field's expander nested, and Streamlit refuses that.

Effect: the script stopped in the sidebar. The canvas and the chat never rendered, for every visitor past the welcome screen, from `df38fa5` until this fix — four commits on `main`.

Why no test caught it: every dashboard test hands its module a fake `st`, and the fake `expander` is a null context manager. It enforces none of Streamlit's layout rules, so the suite passed in full against a dashboard that could not render. `app.py` itself was exercised by hand. The thing asserting correctness was not the thing that runs.

Fixed: - `_key_section` and `render_upload_panel` take `boxed` (default `True`). Unboxed, the label is a caption over the field, with no expander of its own: "Your network" is already the same affordance one level up. No new wording. - The sidebar passes `boxed=False`; the welcome screen keeps the default. - `tests/test_app_smoke.py` boots `app.py` through AppTest for the welcome screen, the demo and a built network, and asserts nothing raised. Against the unfixed code the demo and built cases fail. It is the guard for the whole class, not only this instance: any future layout rule the fakes do not know about fails here. - `tests/test_upload.py` asserts the unboxed panel opens no expander, and that the boxed default still does.

D-67 — A pasted key printed a Streamlit warning on the page (closed 29 Sep)

Found: 29 Sep, reading the key field's `on_validated` path. `app.py` passed `on_key=st.rerun` to the sidebar's upload panel and `welcome.py` passed `on_key=on_ready`, which is also `st.rerun`. `key_field._paste_handler` calls `on_validated` from inside the text input's `on_change` callback.

Effect: every valid paste, on the welcome screen or in the sidebar, rendered *Calling st.rerun() within a callback is a no-op.* as a warning on the page. Otherwise harmless: Streamlit reruns after a callback anyway, which is why the redraw and the retry offer still appeared.

Why low: nothing a visitor did was lost or misreported. It is a warning in a place they can read it, on the screen that asks a stranger for an API key, where anything that looks broken costs trust.

The general case: Streamlit's callback rules have produced defects across P2.8c, P2.8d and now P2.9b. A widget key may be written only in a callback or before its widget exists; a callback already causes a rerun, so `st.rerun()` inside one is a no-op — and it says so on screen. Fake `st` doubles enforce neither rule. Only AppTest does. D-66 is not a sixth instance — it broke a layout rule, not a callback rule — but it reached `main` through the same testing gap.

Fixed: neither call site passes `on_key`; the redraw it asked for already happens. `welcome.py` says why at the call, since dropping it there looks like a lost feature. `key_field.render_key_field`'s docstring says `on_validated` must not call `st.rerun()`. `tests/test_app_smoke.py` pastes a valid key through AppTest on the welcome screen and in the sidebar, with only `validate_key` replaced, and asserts no warning reached the page. Both fail against the unfixed code on exactly that warning.

D-68 — The project key on Community Cloud (closed 29 Sep: not a defect as reported)

Reported: 29 Sep, as High and blocking P2.9. Community Cloud has no shell, so secrets arrive as `st.secrets`. `resolve_key()` and `key_source()` read only `os.environ`, so the deployed app would find no project key, and every visitor would get the keyless build. The proposed fix added an `st.secrets` read to both.

Investigated: `streamlit run`, which is how Community Cloud starts the app, calls `secrets.load_if_toml_exists()` at server start (`web/bootstrap.py`). That copies every root-level string secret into `os.environ` before `app.py` runs. Run against a `secrets.toml` with the shell variable unset, `key_source()` returned `KEY_PROJECT` with the code unchanged. The environment read already sees a Community Cloud secret, as long as it is at the root.

Why the proposed fix was not applied: `st.secrets.get()` with no secrets file calls `st.error("No secrets files found...")` before raising, and does not remember the failure. Every `key_source()` call on a keyless run with no secrets file would have put a red error on the page — D-67's shape, with an error instead of a warning. AppTest substitutes its own secrets object, so the smoke tests would not have seen it.

What was real: - A secret entered under a `[section]` is not copied into the environment, and the app then finds no key. That is the deploy constraint, now stated in the README and pinned by a test. - The app never loads `.env`, so a local `streamlit run` without the key in the shell is keyless — the likely source of the grey screen that prompted this. - `.streamlit/secrets.toml` was not gitignored.

Disposition: no change to `keys.py`. `tests/test_keys.py` loads a `secrets.toml` the way the server does and asserts a root-level key is `KEY_PROJECT` and a nested one is `KEY_NONE`, so a Streamlit upgrade that changes the mechanism fails in the suite rather than on the deploy. The README says where the key goes, locally and on Community Cloud. `.gitignore` covers `.streamlit/secrets.toml`.

D-69 — The classifier is not deterministic, and nothing said so

Found: while clearing `data/cache/claude_classification.json` of real job titles before the public repo. Rebuilding the demo from the cleared cache gave a different answer to the same synthetic file, and the cache had been hiding that since P2.9a: every rebuild since read cached classifications, so the classifier looked stable because it was never asked twice.

Measured, 29 September. Five builds of `data/synthetic/Connections.csv`, identical input, locked config E, same model, cache cleared between each:

Run	Needs review	Share
1	97	21.9%
2	106	24.0%
3	114	25.8%
4	106	24.0%
5	124	28.1%

Range 97–124: 27 people, 6.1 percentage points, mean about 24.8%. A column diff between run 1 and the table shipped on 24 September showed 68 of 442 titles receiving a different `soc_major` — 15.4% — with `role` byte-identical, and the same count of 97 needing review in both. Coordinates and all five centrality columns were unchanged, so the layout is fully seeded and the variation is entirely in the classification. One pair was diffed; how far any two builds differ in general is not measured.

What this invalidates.

The demo's 21.9% was the lowest of five draws, not a property of the synthetic file. Any statement that the demo understates the real 29.0% "by about 7 points" is wrong: across these runs the gap is about 1 to 7 points.

> **Correction note — 6 October 2026 (D-69).** "The demo's 21.9%" above refers to the demo shipped on 24 September, which was built before the cache was cleared and was not one of the five runs. It matched the lowest run's count (97 of 442) while placing 68 titles in different groups from that run. It matched that run's figure; it was not that run. The demo was rebuilt on 29 September and ships at 124 of 442 (28.05%), which is run 5, the highest of the five draws. The date is recorded in `data/demo/provenance.json`; the run is identified in this entry's commit (`95d35e6`). The measurement, the table and the conclusions are unaffected. Only the sentence identifying which artefact carried 21.9% was loose.

The P2.9a calibration spent two pre-registered re-tunings moving a figure whose run-to-run spread (6.1pp) is larger than the miss that stopped the process (2.1pp outside a 24–34% band). Read the way that record reads its own figures, to one decimal, four of these five runs land in band with no tuning at all; two of those four are 106 of 442, which is 23.98% unrounded. The process was followed correctly; the instrument was noisier than it assumed. `docs/p2_9a_record.md` keeps its record and gains a dated note.

What this does not invalidate. The real network's 29.0%, the 60–64% classifiability ceiling and the blind-100 evaluation are all single runs and carry the same kind of uncertainty, but none of them moves far enough under a spread of this size to reopen a decision. Config E stands. Abstention over fabrication stands — if anything this strengthens it, since a classifier that answers differently on re-asking is exactly one that should decline rather than guess. The ceiling's "three convergent ways" survive: only the third, Config E's 63.5% on the blind 100, involves the classifier, and it is a single run with an unmeasured spread. The first two — inter-annotator agreement and a labeller's self-inconsistency — are human and unaffected, and the convergence argument stands on them; a noisy classifier landing near a human ceiling is still evidence for the ceiling. `docs/project_state_decision_log.md` and `docs/project_status_refined_p2.md` carry a dated correction note saying so.

Disposition: not fixed. Non-determinism is a property of the model, not a defect in this code; D-42 came out of the same adaptive-thinking behaviour. What was wrong was stating point estimates as if they were stable. The fix is in the wording, here and in the README, and in never quoting a classification figure again without saying it is one draw.

Next step, not done: re-running the blind 100 three times would put an error bar on the 63.5% precision figure for about the cost of these five runs. Worth doing before any public claim rests on it. Until then its spread is unmeasured, and at 100 titles sampling alone may exceed the 6 points seen here.

D-70 — The profile comparison attributed a difference to the description (closed 29 Sep)

Found: 29 Sep, following D-69 into `self_enrichment.py`. The module docstring called the comparison "the one place the product can test [the ceiling] directly: give the same classifier more of the same person's words and see what changes". The on-screen sentence at line 217 read *"Your title alone produced no group. With your description, the classifier assigns {soc} {name}."*

The confound, which predates D-69: it was never the same classifier asked twice. The enriched reading goes through `self_enrichment.SYSTEM_PROMPT`, which adds rules 2, 3 and 7 — aspiration is not occupation, study is not occupation, the text is data not instructions — because it reads prose. The title-only reading goes through the title tier's prompt. A perfectly deterministic classifier could give a different answer because

it was asked a different question. That was a design choice, and a right one; the panel should not have implied otherwise.

The noise, from D-69: each reading is one call, `use_cache=False` on the title side and no cache on the other, to a classifier that answers differently on re-asking. Pressing "Compare the two readings" again can give a different result.

Effect: six of the eight wordings named a cause. The description "assigned" a group, "made the classifier less sure", made it decline, settled a null, "did not resolve it", or — in the agreeing case — "did not change the answer". Noise makes that attribution uncertain; the prompt difference makes it systematically confounded. Only the both-confident-null and differing-groups wordings were already neutral.

Not borrowed: D-69's 15% was measured on the batched title tier over the demo, with a different prompt and a cache. It says nothing about this path, and quoting it here would repeat D-69's own mistake.

Fixed: every branch reports what each reading says and names no cause, then ends with one of two caveats: *"The two readings use different prompts and each is a single answer, so a difference between them may not come from your description"*, or its agreeing form, *"...so their agreeing does not show your description made no difference."* Both causes, one sentence, neither quantified. The module docstring now calls the comparison "a place to see what changes, not a test of it". `tests/test_self_enrichment.py` pins the caveat on every branch, bans the causal phrasings outside it, and asserts the caveat carries no figure.

Next step, not done: measure this path's own spread. The CLI (`python -m src.self_enrichment --title ... --about-file ...`) runs `compare()` without the dashboard. One real title and About, a handful of runs. If each reading is stable under its own prompt but the two differ consistently, the difference is the prompt, and the remedy is prompt work. If the readings themselves vary, the remedy is repeat calls with a stated agreement. Different remedies, so worth knowing which before building either.

D-71 — "never saved" was not true of the classification cache (closed 6 Oct)

Found: 6 Oct, during P2.9 step 1, verifying the clean-venv install. A keyed build of the author's own 442-person export, run through the upload path, left `data/cache/claude_classification.json` modified: 379 entries to 744, so 365 real job titles from a real network were written into a tracked file.

What was written. Each entry holds a title and the model's one-line `why`. Employers were not written: config E sends `(title, "")` (`classify_pipeline.USE_COMPANY = False`), so every key ends in `||`. That does not make the write harmless, because a rare title can identify a person by itself.

Mechanism. `build_network` takes two adjacent cache parameters with the same type and the same default. The default means opposite things for the two:

```
cache_path: Path | str | None = None,          # -> the shared file on disk
embeddings_cache: Path | str | None = None,    # -> in memory, writes nothing
```

The docstring gives the privacy reasoning for the second and not the first:

> `embeddings_cache` defaults to None, which computes in memory and writes nothing: a raw export is real people's names, and the session build does not leave their vectors on a shared disk.

That reasoning applies unchanged to the classification cache, which leaves their job titles instead of their vectors. `cache_path` reaches `count_uncached_titles` and `classify_people` unmodified, and since D-64 `_resolve_cache(None)` resolves to the default path. `upload.py` never passes one and contains no reference to a cache at all.

A second writer. `retry_unclassified` has the same default. The retry handler in `app.py` passes no `cache_path` either, so retrying the people a build could not classify writes their titles to the same file. A fix to `build_network` alone leaves this path open.

This is D-64's deferred question, one layer up. D-64 recorded that `None` means "no cache" in `embeddings.py` and "use the default" in `claude_classifier.py`, and said that two opposite meanings for

one value in adjacent modules are a trap of their own. That sentinel question was left open. It has now turned up inside a single function signature, between two parameters a reader would assume behave alike.

The suite asserts the defect. `tests/test_cache_isolation.py::test_a_classifying_build_with_no_cache_path_wr` checks that a build with no cache path writes to the cache. D-64 changed *where* the default writes during tests. It never asked *whether* a session build should write at all. The fix reverses that test; a new test added beside it would leave the old assertion standing.

What it contradicts. The welcome screen's first paragraph, the first thing a visitor reads: "Your export is read in memory and never saved." The upload panel and the key field make the same promise. On a deployment the data is not even the visitor's own. It belongs to their connections, who consented to nothing.

Secondary effect. `count_uncached_titles` reads entries that earlier visitors left. The cost quoted in the retry offer therefore depends on who used the app before, and that figure is the one the `uncached_titles` hoist exists to make honest. It also tells a visitor something about other visitors: a file containing a single title shows whether anyone else's network contained it.

Blast radius. Community Cloud's disk is ephemeral across restarts, so the file does not persist indefinitely. Within one container's life it collects every visitor's titles.

Why it was caught. The classification cache is tracked, so the write showed up as a dirty `git status`. Had the file been gitignored like the embeddings cache, nothing would have shown. The detection was incidental, not designed.

Recovered: `git checkout -- data/cache/claude_classification.json`. The entries were never staged, never committed, and never left the machine.

Interaction with D-72. Today the shared cache is also the only thing that keeps a disconnected build's answers. Fixing this entry removes that, so the fix makes D-72 worse (see there).

Fixed: - `claude_classifier._resolve_cache`, `_load_cache` and `_save_cache` accept a dict as the cache itself. `_load_cache` returns it without copying it, so answers accumulate in the caller's mapping, and `_save_cache` writes nothing for it. `count_uncached_titles` and both classify paths go through `_resolve_cache`, so they all inherit the change. `_partition_cached` already took a dict. - `classify_pipeline.classify_people` no longer passes a mapping through `Path()`. - `build_network` and `retry_unclassified` both treat `None` as a fresh dict, created per call rather than as a mutable default argument. The shared file is used only when a caller passes a path. `tools/build_demo.py` does, and `tests/test_build_demo.py` pins it (`5bb711d`). - `session_build.classification_cache(store)` holds one mapping per session. The upload panel and the retry handler in `app.py` both pass it. Start over keeps it, because it is keyed by title, not by row. The one-hour expiry clears it along with the rest of the session. - The `build_network` docstring now says that a `classify=False` build (`tools/diag_canvas.py`) reads an empty mapping by default, so its uncached figure counts every title. - Tests: `tests/test_session_cache.py` (10), plus one for the retry handler in `tests/test_app.py`. The D-64 isolation test is reversed in place, as this entry said it should be. Seven mutations, one per piece removed or reverted, each fail at least one test. 666 tests pass.

Where the fix departs from the drafted spec. - The draft changed only the coercion in `retry_unclassified`, so its `None` still resolved to `CACHE_PATH`. That is the second writer named above. It now gets the same private default, and `test_a_retry_with_no_cache_path_writes_no_file` fails against the draft. - The `_resolve_cache` docstring said "job titles and employers". Config E sends no employer. - The draft's comment in `classify_pipeline` said `Path()` would turn a mapping "silently" into a filename. In fact it raises `TypeError`, so the coercion would have failed every session build loudly. - `classification_cache` lives in `session_build.py`, beside `BUILT_KEY`, rather than in `session.py`, which is only the store guard.

Found while testing, not part of this fix. Four existing tests reach the real Anthropic client whenever `ANTHROPIC_API_KEY` is set in the shell. They are `test_upload.py::test_a_large_upload_is_warned_about_but_not_refu` `::test_a_new_build_clears_a_pending_offer` and `::test_the_panel_says_what_it_did_not_use`, and `test_welcome.py::test_uploading_from_the_welcome_screen_builds_and_moves_on`. A probe

that refuses `_default_client` found the same four at `ec829ef`, so they predate this fix. D-62's sweep could not see them. With the key unset or set to a fake value, the call fails into the fourth state, and these tests still pass.

D-72 — nothing to watch, and a disconnect does not stop the spending

Measured: 6 Oct, with the same export and run as D-71. It has 442 people and 384 distinct titles, 379 of which were uncached against the synthetic-only cache. A keyed build through the upload path took about five minutes. Titles are deduplicated before batching, so the number of distinct titles sets the work, not the number of people: 379 titles in 19 batches of 20. Those 379 titles were stored under only 365 cache keys. That is because the run deduped on the raw title string, but the cache keys on the normalised title, so 14 of the titles sent were variants of other titles in the same run (D-73). The cost was about $0.36 at the measured $0.00095 per title. The pre-flight estimate prices it at $0.45, using `ASSUMED_COST_PER_TITLE`, which is well inside `MAX_SPEND_PER_GENERATION`. The duration is what the work costs. The defect is what surrounds it.

Measured on the deployed app, 7 Oct: on Community Cloud, an export of about 305 people took about 3 minutes, behind the spinner only. That is not a slower rate than the local run: about 0.6 s per person, against about 0.7. What is worse is the setting. This is a stranger's first impression, on a shared container.

No progress. Five minutes behind a spinner is past the point where a visitor assumes the app has hung. The place to report progress exists: the batch loop in `classify_claude_outcome`, which already checks the meter between batches on the project key. `provenance["uncached_titles"]` already supplies the denominator. 19 batches is fine enough to show progress.

What a disconnect does. This is read from the code, not observed. Streamlit 1.41.1 stops a script cooperatively: `ScriptRunner.request_stop` takes effect at the next interrupt point, meaning the next Streamlit call. The batch loop makes no Streamlit calls. A closed laptop or a dropped connection therefore does not stop the build. Every remaining batch is still sent and paid for, and the result goes to a session nobody is watching. - **Today, the answers survive, through D-71's leak.** The `finally` in `classify_claude_outcome` saves every answer to the shared cache, so if the visitor uploads again, those titles cost nothing. The visitor loses the network on screen, not the classifications. - **A process restart loses the work as well.** A Community Cloud reboot or a crash runs no `finally`. The answers are lost, and so is the daily ledger, which is ephemeral there. The visitor paid and received nothing, and there is no record on either side. On the visitor's own key, the money shows up on their Anthropic bill and nowhere in the app.

Fixing D-71 makes this worse. Once the session build keeps its cache in memory, a disconnect also loses the classifications. Any recovery built for this entry must be scoped to the session or to the export, never to the shared file. Otherwise it reintroduces D-71.

Correction note — 6 October 2026 (D-71). After D-71's fix: The first bullet above describes the code before that fix. A session build now writes its answers into the session's own mapping, batch by batch. They survive exactly as long as the session does. A visitor who reconnects to the same session and uploads again pays nothing for them. A new session, or a process restart, starts empty. This is read from the code, not observed; how long a dropped session can still be rejoined is Streamlit's reconnect window.

Blocks step 7 (the gate), not step 4.

D-73 — the same key, classified more than once in one run

Found: 6 Oct, verifying the D-71 commit that makes `build_demo.py` name its cache. A demo rebuild from a fully populated cache reproduced every row but one: "Vice President - Global" moved from confidence 0.75 to 0.95, in the same group. Three synthetic titles normalise to the key `vice president||`. The original build sent all three, and the cache kept the last answer written.

Mechanism. `_partition_cached` checks `items` against the cache only. Two items with the same `_cache_key` are both missing from the cache when the check runs, so both go into `todo`. `classify_people` dedupes titles but not keys, and the key normalises more aggressively than the title does.

Three consequences, from least to most serious.

1. Duplicate spend. In the synthetic export, 4 keys cover 9 titles. In the author's real export, 379 uncached titles collapse to 365 new keys.
2. Two people whose titles normalise to the same key can receive different answers in the same session, from the same input, with nothing on screen to say so. This is D-69's non-determinism appearing *within* a run rather than between runs.
3. A rebuild from a full cache reads the cached classifications but does not reproduce the shipped demo exactly. The argument for tracking the cache, so that a rebuild reproduces the demo, holds for the cache but not for the node table. This time the difference was one confidence value: the group and the review count (124) were unchanged.

Trap for whoever fixes it. The deduplication happens in two places, and both must change together. `count_uncached_titles` also counts raw titles. Today that is honest, because the pre-flight estimate and the retry offer price exactly what is sent. If keys are deduplicated in `_partition_cached` alone, both figures will overstate the cost. That is the figure the `uncached_titles` hoist exists to keep honest.

Not fixed here. The fix is to check against the cache and the pending batch together. That means deciding what a second item with an already-pending key gets: the same answer, or its own call. The first is cheaper and self-consistent. The second is what the code does today, by accident. That decision belongs with the classifier, not inside a deployment slice.

Disposition for P2.9: logged, not fixed. It costs money and it is untidy, but it puts nobody's data anywhere and contradicts no sentence on screen. The one written claim it did contradict, in `docs/p2_9b_closure.md` §3.4, was softened before that record was committed.

D-74 — the suite spends money, and nothing says so (closed 6 Oct)

Found: 6 Oct, while verifying D-71's fix. Four tests reach the real Anthropic API when a key is present in the environment:

```
test_upload.py   test_a_large_upload_is_warned_about_but_not_refused
                 test_a_new_build_clears_a_pending_offer
                 test_the_panel_says_what_it_did_not_use
test_welcome.py  test_uploading_from_the_welcome_screen_builds_and_moves_on
```

A probe that refused `claude_classifier._default_client` found these four at `ec829ef`, so they predate the D-71 fix. A second probe refused the SDK's own send (`SyncAPIClient.request`). That catches every route, and found the same four with the key set and none with it unset.

Why nothing caught it. The four pass under every environment: - Key unset: the panel builds keyless and nothing is called. - Fake key: the call fails, and the build lands in "not classified yet". - Real key: the tests pass, having spent money.

None of the four asserts anything that would tell these cases apart. D-62 checked that tests do not depend on the ambient shell and could not see this, because the outcome under every environment it tried was "passes".

This is the D-64 family, inverted. D-64 was a test reaching a real artefact and leaving a trace: a fabricated entry and a dirty `git status`. Here a test reaches a real service and leaves no trace at all. There is no file, no output, nothing to notice. The only signal is the bill, arriving later and attributed to nothing. D-64 was caught because the cache happened to be tracked. This was caught only because D-71 made someone look at what talks to the network.

Where the guard goes. The guard does not go at the environment variable. Blocking a missing key only reproduces the case that already passes silently. It does not go at `claude_classifier._default_client` either. There are three `_default_client`s (classifier, assistant, summaries), and `keys.validate_key` builds its own client. Nor does it go at `anthropic.Anthropic` construction: `test_resolve_client_threads_the_store` builds a real client on purpose, and building one costs nothing. Sending a request is what costs money. So the guard goes at the SDK's send, `anthropic._base_client.SyncAPIClient.request` (and the async twin). That is a method on the base class, so it catches every client, however it was built or imported.

Next step: an autouse conftest fixture that makes that send raise, so a test reaching the API fails loudly. The four then fail until each is given a fake client. This is the same shape as D-64's guard: make the safe path the default, and make forgetting it an error rather than a cost. Fix the four one at a time, and read what each starts asserting. A test that has been passing through the not-classified path may never have exercised what its name claims. Read `test_the_panel_says_what_it_did_not_use` first: "what it did not use" sounds like it depends on whether classification happened, which is exactly the axis that was unconstrained.

Cost incurred: cents, across this session's suite runs.

Fixed: - `tests/conftest.py` has an autouse `_refuse_real_api` fixture that patches `SyncAPIClient.request` and `AsyncAPIClient.request` to raise `RealAPICallRefused`. It also records every attempt and fails the test at teardown, so a refusal swallowed by a broad handler still fails. `RealAPICallRefused` subclasses `BaseException` because the chat, the summaries and the retry handler all catch `Exception` and show a degraded result instead. `tests/test_api_guard.py` sends through the chat with no client and asserts the refusal escapes. That test fails if the refusal is made an ordinary `Exception`. - The four tests were fixed one at a time. None of them asserts anything about classification, so each now pins that axis rather than leaving it to the shell: - `test_the_panel_says_what_it_did_not_use` runs both sides, keyless and classified with a fake client, and asserts the intake note on each. It also asserts which side ran. "What it did not use" turned out to be the intake (rows skipped, columns not read), which never depended on classification, so the run on both sides shows that. - `test_a_large_upload_is_warned_about_but_not_refused`: keyless. The warning comes before the build. - `test_a_new_build_clears_a_pending_offer`: keyed, with a fake client. `key_field` makes an offer only when a key is pasted over a keyless build, so a pending offer means a key is present and the next upload classifies on it. The keyless version tested a case that cannot occur. - `test_uploading_from_the_welcome_screen_builds_and_moves_on`: keyless, as a first visit is. - 668 tests pass with the real key set, with it unset, and with a fake value. With the guard in place, the run with the real key sent nothing. - The red step, kept here rather than as a failing commit on main. With the guard alone, the suite showed 4 failed plus 4 teardown errors, and no others: `test_a_large_upload_is_warned_about_but_not_refused`, `test_a_new_build_clears_a_pending_offer`, `test_the_panel_says_what_it_did_not_use` and `test_uploading_from_the_welcome_screen_builds_and_moves_on`. To reproduce it, revert the four fixes (`git checkout f61cff3 -- tests/test_upload.py tests/test_welcome.py`) and run with `ANTHROPIC_API_KEY` set to any value. A fake key works, because the guard refuses before anything is sent. With the key unset, the old four build keyless, never send, and pass, which is D-74's own blind spot. All three runs were re-checked on 6 Oct against `998179e`.

Where the fix departs from the entry's Next step. The Next step said the four would fail "until each is given a fake client". Only one was given a fake client alone, and one runs both sides. The other two are pinned keyless instead. A fake client removes the cost but not the axis: whether those tests classify would still depend on the shell, just without paying. Each test now fixes the side its claim belongs to.

D-75 — the README said deleting `data/cache/` is always safe (closed 6 Oct)

Found: 6 Oct, while adding D-71's paragraphs to the README's Caches section. Priority low.

Defect. The README said `data/cache/` "is gitignored and everything in it is recomputable, so deleting the directory is always safe". In fact `.gitignore` ignores `data/cache/*`, then un-ignores four files: `claude_classification.json`, `sweep_results.csv`, `sweep_silhouette.png` and `umap_sweep_results.csv`. Anyone who followed the instruction and then rebuilt the demo would have paid for 384 titles. Given D-69, they would also have got a fresh draw and a different demo.

False from the day it was written. The sentence arrived in `a2cd0a8` (29 Sep). `.gitignore` had un-ignored the classification cache two days earlier, in `8d2034d` (27 Sep).

Fixed in `197102b`: the section names the four tracked files and says to restore them with `git checkout -- data/cache` before rebuilding anything.

D-76 — tracked files derived from the real network, invisible to every `.gitignore` check (closed 6 Oct)

Found: 6 Oct, during P2.9 step 3 (the real-data sweep). They were found by listing the tracked files under `data/` (`git ls-files data/`), not by reading `.gitignore`. Priority medium.

What the sweep found. Five tracked files under `data/` were derived from the real network. None is person-level, so none discloses an individual: - `data/labels/cluster_top_terms.png`: the TF-IDF cluster labels, drawn as a plot. It renders real job titles as text in an image, which is the closest this repo has come to publishing the real network. Added in `b283c03` (7 Jul) and refreshed in `5398bcf` (9 Sep). - `data/graph/graph_metrics.csv`: twelve aggregate graph metrics (`d55cea5`, 7 Jul). - `data/cache/sweep_results.csv`, `sweep_silhouette.png` and `umap_sweep_results.csv`: the Phase 1 HDBSCAN and UMAP parameter sweeps over the real embeddings (`10d0a28`, `cff9532`, 2–3 Jul). Each row is one parameter setting, with cluster counts, outlier share, and silhouette or trustworthiness. They contain no titles and no names.

`.gitignore` does not untrack anything. A rule stops untracked files from being added. It has no effect on a file already in the index. Every real-data check before this one reasoned about what the rules exclude, and these files were in the index before the rules existed. The August review is the clearest case. `acb22e1` (18 Aug, "gitignore real-network artefacts", with personal data purged from history by `git-filter-repo`) ignored `data/labels/cluster_labels.csv`. The plot of the same labels, in the same directory, stayed tracked, and was refreshed three weeks later. Reading the rules could never have surfaced it. Only asking the index can.

Why it took this long. Each earlier check answered "what do the rules keep out?" None asked "what is in the index?" The two questions agree only for files added after their rule.

Disposition: - The PNG is untracked (`git rm --cached`) and ignored by name in `.gitignore`, beside `cluster_labels.csv`. It stays on disk, and in this private repository's history. The public repository is created fresh, with no history (P2.9 step 4, option 2), so it carries neither. - The four aggregate files stay tracked. They are figures about parameter settings and the graph as a whole, with no title, name or person-level row.

Related, not changed here. `graph.load_nodes()` still reads `cluster_labels.csv` unconditionally (`src/graph.py:111`), and the comment at line 47 still calls it a "tracked input (Day 6)". Only the command-line chain calls `load_nodes` (`graph.py`, `centrality.py`); the app and `build_network` do not. So a deploy does not need the file, but the CLI chain fails on a clone without a locally generated one.

> **Correction note — 6 October 2026 (D-76).** Disposition changed for `graph_metrics.csv`. The disposition above kept it tracked because twelve aggregate statistics disclose nothing about an individual. That still holds. It is now untracked for a different reason, and `data/graph/` is ignored.
>
> It is a Phase 1 output of the HDBSCAN pipeline, which was discarded at Phase 1.5. `src/graph.py` can regenerate it only from `data/cache/embeddings.npy` (the pre-keying file that nothing reads any more), `clustered.csv`, `cluster_labels.csv` and the projection. None of these is tracked, and the script takes no arguments, so it cannot be pointed at the synthetic export. No code reads the file. The two Gate C records cite it as historical observations, and those observations stand without it. It is also not evidence for what the project asserts; it disagrees with it. The methodology documents (`phase1_5_methodology.md`, `methodology_summary.md`, `methodology_maths.md`) report modularity 0.5224 and ARI 0.8985. This file reports 0.3429 and 0.0557. Shipping it would publish a summary of the author's private network, describing a pipeline the project no longer uses, that nobody can reproduce, and that contradicts the published figures. That contradiction is recorded here, not resolved.
>
> **The three sweep outputs stay, and the test that separates them is whether a file is still evidence for something the project asserts.** "It's aggregate, so keep it" and "it's aggregate, so remove it" would otherwise look inconsistent. - `umap_sweep_results.csv` is cited in `src/config.py` as the evidence for the UMAP parameters the projection still uses (D-29). -

`sweep_results.csv` and `sweep_silhouette.png` are HDBSCAN sweeps. They are evidence for the README's claim that the real data has "no density for a clustering algorithm to find": 61–69% outliers at small cluster sizes, and 2 clusters at size 15. That claim is the reason HDBSCAN was dropped.

After this change, 12 files are tracked under `data/`: five O*NET tables, the synthetic export, the two demo files, the classification cache and the three sweep outputs. `data/raw` appears in no commit on any ref.

Correction note — 6 October 2026 (D-76). The note above says this file "disagrees with" what the project asserts and "contradicts the published figures". It does not. The two sets of figures were computed over different data: - The methodology documents' 0.5224 and 0.8985 come from `d55cea5` (7 Jul), computed over the 83 generated profiles of Phase 1 (`n_nodes` 83). - This file's 0.3429 and 0.0557 were computed over the 442-person real network (`8468ed0`).

The collapse was already visible over 430 people on 19 July (`467b189`: 0.3352 and 0.047), and `phase1_5_methodology.md` reports it that day, as its central finding. The disposition stands on the note's other grounds: the pipeline was discarded, and the file can be regenerated only from untracked real-network artefacts. What the methodology documents do lack is recorded as D-77. The commit message of `eb98946` makes the same mistaken claim.

Also checked, 6 October: not a defect. Four tools exist in the working tree but have never been in the repository: `tools/diag_canvas.py`, `diag_chat.py`, `diag_maxtokens.py` and `diag_summary.py`. `.gitignore` has ignored `diag_*.py` by name since `72eec50` (17 Aug). Nothing claimed they shipped. This came to light while the README's tools section was being checked against a fresh clone, where they were missing. It is this entry's lesson inverted. Here, files were tracked that a `.gitignore` review could not see. There, files appear in a directory listing that suggests they ship, and they do not. Both questions are answered only by asking git what it actually tracks (`git ls-files`), or by running from a fresh clone.

D-77 — Phase 1's conclusion stands in two records with nothing saying it fell (closed 6 Oct)

Found: 6 Oct, tracing the figures behind D-76's note. It was raised as "three published documents cite figures nothing supports". The investigation does not support that framing.

What the figures are. These are every committed version of `data/graph/graph_metrics.csv`:

Commit	Date	Nodes	Modularity (HDBSCAN)	ARI (greedy vs HDBSCAN)
`d55cea5`	7 Jul	83 (generated profiles)	0.5224	0.8985
`467b189`	19 Jul	430 (real export)	0.3352	0.047
`8468ed0`	9 Sep	442 (real export)	0.3429	0.0557

ARI 0.8985 means the two partitions substantially agree. ARI 0.05 is barely above chance, which refutes the agreement rather than merely differing from it. The refutation came from moving from 83 generated profiles to the real network. It did not come from D-28's regeneration (`8468ed0`), which only moved 0.047 to 0.0557.

What it is not. - `phase1_5_methodology.md` reports 0.5224 and 0.8985 scoped correctly ("earned on data engineered to be clusterable"). It makes the collapse on real data its central lesson, and it was published on 19 July, the day the 430-node figures appeared. - `methodology_maths.md` states N = 83 wherever it gives the figures.

No published claim about the real network was silently invalidated.

What it is. `methodology_summary.md` and `methodology_maths.md` were written on 9 July and are tracked with PDFs. They describe Phase 1 on the 83 generated profiles, and neither carries a note. - The summary

concludes that, with four lenses agreeing, " 'the structure is real' stops being an assertion and becomes a defensible conclusion". - Its decision ledger lists HDBSCAN as "Chosen".

In the public repo, a reader who lands on either document reads a method the project discarded, presented as the project's method. Nothing there says that on the real network the corroboration fell to ARI 0.047, or that HDBSCAN was dropped at Phase 1.5.

The part of the original framing that holds. `467b189` overwrote `graph_metrics.csv` in place, so from 19 July the tracked file no longer supported these documents' numbers. Their only artefact is `d55cea5`, in this private repository's history. The public repository is created without history, and `graph_metrics.csv` is now untracked (D-76). So in the public repo, 0.5224 and 0.8985 rest on no artefact at all.

Also seen. `phase1_5_methodology.md` says the 83 profiles were drawn from "eleven hand-designed archetypes". `methodology_summary.md` says "seven distinct professional" archetypes and "all 7 archetypes recovered". Not reconciled here.

Next step: a dated note at the top of each of the two Phase 1 documents, in its own commit, `.md` and `.pdf` together. The note should say: - the figures are over the 83 generated profiles - on the real network ARI was 0.047 (430 people) and 0.0557 (442) - HDBSCAN was discarded at Phase 1.5, with a pointer to `phase1_5_methodology.md` - the artefact behind the figures is in the private history only

Fixed: a dated note at the top of each document, carrying everything the Next step listed: - the figures were measured on 83 generated profiles - the real-network figures: ARI 0.047 and HDBSCAN modularity 0.3352 over 430 people, and 0.0557 and 0.3429 over 442 - HDBSCAN was discarded at the Phase 1.5 close-out on 19 July - a pointer to `phase1_5_methodology.md` - the source run survives only in the private history

Both documents are otherwise left as written, and each `.md` was committed with its `.pdf`. The notes were aimed at what each document actually says, which was not quite what the brief for them said: - `methodology_summary.md`. Its decision ledger is §11, not §12. It does not end on "the structure is real"; that is §10. It ends at §13, on an engine "complete and validated" that "everything downstream reuses". The note addresses §10, §11 and §13. On §13 it says what is true: the embeddings, projection, graph and centrality were carried forward; the clustering and its labels were not. - `methodology_maths.md` has no "Chosen" framing. What misleads there is a headline table titled "(real values)", introduced as metrics "from the current build". "Real" there means measured, but it reads as the real network. The note says which it is.

Still not reconciled: the archetype count (eleven in `phase1_5_methodology.md`, seven in `methodology_summary.md`), noted above.

The PDFs were rebuilt by pandoc with pdfTeX. The summary uses its own front matter. The maths document has none, so it uses the summary's settings, A4 with 2.5 cm margins. The earlier PDFs' producer and page size could not be read, so their appearance may differ.

D-78 — the decision log named five real connections (closed 6 Oct)

Every earlier real-data check looked where real data is supposed to live: `data/raw/`, the `.gitignore` rules, and `git ls-files data/`. This was prose in a document, in `docs/`, where nobody thought to look because documents are not data. It is the third variant today of the same lesson. D-76 found files that a `.gitignore` review could not see. This found data in a directory that a `data/` review could not see.

Found: 6 Oct, P2.9 step 3, by the full-history scan the step recommends. The scan took the 441 full names in the real export and searched every blob on every ref, 155 commits, for whole-word matches, excluding `data/raw`. Exactly one file matched: `docs/project_state_decision_log.md`, lines 684–685, in entry P3.11, where five real connections were named. None of the five names occurs in the synthetic export, so they are not coincidences with Faker output. The text had been there since P3.11 was logged (`a52af9b`, 8 Sep), in the document's 101 commits since. It was pushed to the private origin. The tracked PDF was rebuilt on 22 Sep (`2e4b903`), after P3.11, so it almost certainly carried the names too. That is inferred from the dates, not verified: the old PDF's text cannot be read with the tooling used here.

Second pass: surnames alone. Whole full names miss a document that writes "Dave" where the export says "David". A surname-only pass covered every tracked text file at HEAD, excluding the Faker-generated synthetic and demo data and the public O*NET tables. It found 27 hits on 14 surnames. Apart from the five, they were: - the author's own surname, in author lines - two ordinary English words - test-fixture and docstring names

Every fixture whose surname a real connection shares was then checked against that real person's first name, both exactly and by its first three letters. None matched. So nothing beyond the five.

What the scans cannot see. A person mentioned by first name alone, or by a nickname that shares no prefix with the exported name ("Bob" for "Robert"), would not match either pass. First names alone are too common to scan meaningfully. That is the residual a third pass would have to address, by reading rather than matching.

A near-miss during the fix. The first script written to show the offending lines with the names masked printed one line per name, masking only that name. The other four appeared unmasked in the session output. The output stayed local and the data is the author's own. But a redaction script that prints the thing it is redacting is a real pattern. The fix was to build the mask over every name token before printing anything, and to assert that no token survived. The working list of real names written to the scratchpad for the scan was deleted afterwards.

Fixed: the names were redacted in place in both the `.md` and the `.pdf`. P3.11 now reads "a tight cluster of five connections, all classified into the same group". Neither the occupation nor the group is named: with five people removed from a document that describes the network elsewhere, the group plus the count is a narrower pointer than it looks. A dated note under the entry says that five names were removed, without restating or characterising them.

A dated note cannot fix a disclosure, so this is the one case the dated-note convention cannot cover. The convention keeps a record's history legible, and assumes that what is being corrected is a claim. Leaving a disclosure visible while annotating it would be the worst of both. The public repository is created without history (step 4, option 2), and that is what makes an in-place edit sufficient rather than a workaround. The names remain in the private repository's history.

The rebuilt PDF was checked by decompressing its text streams. None of the ten name tokens from the redacted lines is present, and the same extractor does find ordinary words in it. The PDF is now built by pandoc with pdfTeX, from the document's own front matter (geometry, contents, font size). Its earlier producer is unknown, so its appearance may differ.

D-79 — the command-line chain has not run since 24 September (closed 6 Oct)

Found: 6 Oct, while checking the README's claim about running modules. Each chain module was run in a fresh clone of `HEAD` with the API key unset. Most failed, as expected, on artefacts the repository does not carry. `src.ingestion`, `src.embeddings` and `src.classify_pipeline` failed differently, with `TypeError: load_linkedin_profiles() got an unexpected keyword argument 'path'`. The same call fails in this repository with the real export present.

Mechanism. `719d64f` (24 Sep, P2.9b upload) renamed `load_linkedin_profiles`'s first parameter from `path` to `source`, so that it could take an upload as well as a path. `src/ingestion.py:23` still passed `path=csv_path`. `ingestion.load_profiles()` is where every command-line module starts: `embeddings`, `classify_pipeline`, `ingestion` itself, and `graph.load_nodes()`, which `graph` and `centrality` run. So the whole chain has been unrunnable for twelve days, even with every artefact on disk. It is the only stale caller: `build.py` and `tools/measure_cost.py` pass the argument positionally, and `linkedin.main()` passes none.

Unaffected: the app. The upload path and `build_network` call `load_linkedin_profiles` directly, with the new parameter.

Why nothing caught it. No test calls `load_profiles()`. `tests/test_runtime_imports.py` imports `src.ingestion`, which confirms the module loads, not that it works. That is the shape of check that D-66 and D-67 defeated. The quarantined `tests/test_ingestion.py.broken` is a Phase 0 stub of functions that

no longer exist (`bc614af`, 18 Aug), so it had no bearing. A rename landed, the suite stayed green, and the only consumer of the renamed parameter was a path nothing exercises.

Next step: pass the argument positionally, and add a test that calls `load_profiles()` on the tracked synthetic export. Then rerun the chain with the real data and the local artefacts present, to see whether it gets further and hits a second stale call. That would be part of this defect, not a new one.

Fixed: `ingestion.load_profiles()` passes the path positionally. `None` still falls through to `config.LINKEDIN_CSV`.

`tests/test_ingestion.py` calls `load_profiles()` on the tracked synthetic export, both with a path and through the configured default. Both tests fail against the old keyword. What they protect is not `load_profiles` as such but the common prefix of every command-line module. `embeddings`, `classify_pipeline`, `ingestion` and `graph.load_nodes()` (and through it `graph` and `centrality`) all start there, so one call exercises the entry to the whole chain.

The chain gets the whole way through. All ten modules were rerun in a scratch copy of the working tree, with the real export and the local Phase 1 artefacts present and the API key unset. Nine exit cleanly, from `ingestion` through `graph` and `centrality`. `classify_pipeline` stops with `MissingKeyError`, as it should with no key: the real titles are not in the tracked cache. There was no second stale call. The copy's classification cache was unchanged, and the copy was deleted.

Also seen, not changed. With a key, `python -m src.classify_pipeline` on the real export would classify the real titles into the shared, tracked cache. D-71 deliberately left `CACHE_PATH` as the command-line default, so this is by design. But on this machine it is the D-71 write by another route, and the dirty `git status` that found D-71 would be the only sign.

D-80 — the chat allowance exists, and nothing applies it (closed 7 Oct)

Found: 7 Oct. It was raised as "the chat allowance isn't disclosed": a visitor gets ten questions, with nothing saying so until the eleventh fails. Checking where to put the disclosure showed that the eleventh does not fail. No question does.

What exists. `src/dashboard/chat_allowance.py` (`67713b7`, 27 Sep) implements design record §6.6 (`docs/p2_9b_upload_key_tier.md`). It counts ten questions per session on the project key and none on the visitor's own. `Allowance.caption()` already returns the right words: "N free questions left", per session and not per day, and "Free questions used up — add your own key to keep asking". `current()` reads the count and `record_question()` advances it. `tests/test_chat_allowance.py` covers all of this directly.

What is missing. `src/dashboard/chat.py` never calls `current()` or `record_question()`. No caption is shown, nothing is counted, and nothing stops. On the project key the chat is unlimited. Each question sends the whole network, about 9,000 tokens or roughly $0.02 at 442 people (§6.6's figures). The cap existed so that "forty questions from one visitor" would not cost another whole build. On any deployment holding the project key, that exposure is live.

Not charged to the ledger either. Neither the chat, the summaries nor self-enrichment touches a meter or the daily ledger. Only classification does. So the $30-a-day cap bounds classification alone. `docs/p2_9b_closure.md` records "Chat spend | Charges the daily ledger only" among the decisions locked, and that is not true of the code.

Why nothing caught it. The module's tests call it directly. Nothing tests that the panel uses it. It is D-79's shape, and D-66 and D-67's before that: the unit exists and passes, and the integration was never made. The closure record listed the allowance as delivered (`FREE_CHAT_QUESTIONS` 10), because the module and its tests existed.

What the original framing got right. The disclosure belongs beside the question box, next to "What is sent when you ask a question". And the wording must promise only what the code keeps: per session, so "N free questions left", never "today". The caption already says that, so wiring it in discloses it.

Next step: - In the chat panel, read `current()` before accepting a question and show its caption beside the box. - Call `record_question()` on each question answered on the project key. - When the allowance is spent, stop the chat and point at the key field. The network and everything else stay usable, as §6.6 specifies. - Test it through the panel, not the module, so the integration itself is what is checked. - Decide whether chat, summary and enrichment spend should reach the daily ledger. - Correct the closure record's "charges the daily ledger" line with a dated note, in its own commit.

Fixed, following the design record as amended on 7 Oct (`e5777e0`): one ledger, three sources, and one question count with a single source. - **The chat panel.** On the project key it shows the allowance caption beside the box ("N free questions left"). A question counts only once it has been answered, following `record_question`'s own rule. When either the ten questions or the day are spent, the box is removed, any question already submitted is dropped unanswered, and a caption says why and points at the visitor's own key. The visitor's own key is neither counted nor charged. - **Chat, summaries and self-enrichment** all charge the daily ledger through `metering.LedgerClient`. It wraps whatever client the path would have used, checks the day before each call, and records the response's usage after. All three paths use only `client.messages.create`, so one wrapper meters all of them. None of them touches the generation meter, and only the chat counts questions. - **On a spent day,** the summaries panel says so, and the enrichment panel catches `BudgetExceeded` before the generic `RuntimeError` handler. That handler would otherwise have shown "day budget spent: $30.00, cap $30.00". - **The build's notice** for `budget_reached` now ends "Add your own key to classify them now." The recorded phrase ("today's free allowance ran out") is unchanged, because `docs/p2_9b_partial_failure.md` quotes it.

`tests/test_chat_metering.py` (9 tests) drives the panels themselves: - A question through the chat panel uses one question. - The eleventh is refused and points at the key field. - A question that got no answer uses none. - A question is charged to the ledger. - The visitor's own key is neither counted nor charged. - A summary is charged and uses no question. - A comparison is charged, and is refused on a spent day. - When the day runs out, the chat and a build both refuse and point at the key field. Design item 6, end to end.

Nine mutations, one per piece of the wiring, each fail at least one test. 686 tests pass.

Found while testing. `DailyLedger.check()` with no estimate refuses only once spend *exceeds* the cap. A day spent to exactly $30 would have let a wrapped call through, while `budget_spent()` correctly called the day over. The build path never meets this, because its checks always carry a positive estimate. The wrapper now uses the same test as `budget_spent()`, and the spent-day comparison test fails against the old one.

Where this departs from the Next step, and from the brief. - Summaries are produced when a visitor selects a group, not "as part of a build". The conclusion is the same: the visitor did not ask a question. - The budget pointer on the build's notice was not in the Next step. It is what makes design item 6's "points at the key field" true for builds as well as chat. - The ledger-scope decision is recorded in the design record, not here.

D-81 — the spend caps are built, tested, and never constructed (closed 7 Oct)

Found: 7 Oct, reading `spend.py` to wire D-80's chat into the daily ledger. `Meters`, `DailyLedger` and `GenerationMeter` are constructed nowhere outside `src/spend.py` and its tests. `spend.py` (`8d2034d`, 27 Sep) was tested as a unit and never connected to anything that runs.

What it disables. The enforcement code exists and is correct. It never runs, because `meters` is always `None`: - The pre-flight refusal in `build_network` (`build.py:352`). - The per-batch stop in `classify_claude_outcome` (`claude_classifier.py`, the `meters.check` at each batch boundary).

`upload.py` calls `build_network` without `meters`, and `retry_unclassified` has no `meters` parameter at all. So on the project key: - No build is held to $5. - No day is held to $30. The ledger file `data/spend/daily.json` is never written. - Design item 6 can never happen: when the daily budget is spent, upload still works and everyone lands in the fourth state with the key field offered. - D-80 adds the chat, and the summaries and enrichment, which also touch no meter.

The only limit in force is the Anthropic Console's $150 a month. The 7 Oct deployed build (D-72) spent without any record in the app.

What it contradicts. `docs/p2_9b_closure.md` §3.5 says "the pre-flight estimate **refuses rather than half-spends**", and describes the mid-build stop and the file-backed ledger. That is all true of `spend.py`, and none of it is true of the app. The design record's slice 3 is "Both ceilings, enforced".

Why nothing caught it. The same shape as D-79 and D-80, at a larger scale. `tests/test_spend.py` exercises the meters directly, and the build tests inject a `Meters` where they need one. Nothing asserts that the upload panel passes one. Every test of the cap passes a meter in itself, so every one of them passes, and the app never does.

Entangled with D-80. The chat, summaries and enrichment need a `DailyLedger` to charge, and builds need one too. Fixing either means deciding where the ledger is constructed and how a session reaches it. One fix serves both.

Next step: - Construct a `Meters` in the upload panel when the key source is the project's: a fresh `GenerationMeter` per build, and the shared `DailyLedger`. - Give `retry_unclassified` a `meters` parameter, and pass one from the retry handler. - Charge the ledger for chat, summaries and enrichment on the project key (D-80). - Test through the panel: an upload on the project key is metered; a build is refused at the pre-flight when the ledger is spent, and lands in the fourth state; and the retry is metered. - Correct the closure record and the design record with dated notes.

Fixed: - `src/dashboard/metering.py`'s `meters_for(source)` returns a `Meters` on the project key only: a fresh `GenerationMeter` per build, over the shared `DailyLedger`. Otherwise it returns `None`. - The upload panel passes it to `build_network`. - `retry_unclassified` takes `meters` and passes it on to the classifier. The retry handler in `app.py` passes `meters_for(key_source())`. - A metered build that classifies now calls `record_network()`, the ledger's network tally, which had also never been written.

`tests/test_metering.py` (7 tests) goes through the upload panel and the retry handler. On the project key, a build is charged to the ledger and counted. When the day is spent, a build is refused at the pre-flight, sends nothing, still builds, and lands every person in the fourth state with `budget_reached` (design item 6, exercised for the first time). A retry through the handler is metered, and a retry against a spent day sends nothing. On the visitor's own key, neither is metered. Four mutations, one per piece removed, each fail at least one of them. 677 tests pass.

A guard the fix needed. The first suite run after the wiring created `data/spend/daily.json` in the working tree and counted four networks in it. That is D-64's shape, with the ledger in place of the classification cache. `tests/conftest.py` now redirects `spend.LEDGER_PATH` for every test, and a session backstop fails the run if the real ledger changes. The stray file was deleted.

Known limit, not fixed. The ledger is a file on the container's disk, and Community Cloud's disk is ephemeral. A recycled container resets the day's spend, so the $30 is per container lifetime as much as per day. Design §6.5 already names this, and the Console's $150 a month is the guard for it. The 7 Oct amendment to the design record restates it as built: the ledger survives a process restart, but not a recycled container.

The Next step's dated notes are split as planned. The design record was amended before this fix (`e5777e0`). The closure record's notes follow the D-80 fix, so they describe the code once both are in.

Rule, from the third time. This is D-64 arriving by a third route. The classification cache was written by a test that classified without a path (D-64). The cache's default write path was the D-71 leak. And the daily ledger was written the moment it was wired to a panel. The conftest guards now cover two real artefacts, the classification cache and the daily ledger, each with a per-test redirect and a session backstop on the real file. The pattern is established enough to be a rule rather than a surprise: anything with a path and a writer gets a conftest redirect, in the same commit that wires it to the app.

D-82 — the first scripted cut erased an edit made on the public side (closed 7 Oct)

Found: 7 Oct, in the force-push's own output: `+ 2172732...311b4c7 main -> main (forced update)`. The remote was at `2172732`, not at `8b76485`, the commit the local public copy held. `2172732` was "Rename project title in README", made in GitHub's web editor at 12:59 UTC on 7 Oct on top of `8b76485`. It changed one line: `# Skill-Clustered Network Visualiser` to `# Network Visualiser`. The private repo never had it, so the cut, built from private HEAD, overwrote it. The public repo showed the old title again. GitHub still served the commit by SHA, which is how its content was recovered.

Why the review missed it. Every check compared against the *local* public copy: the tree comparison that morning, the script's origin guard, and the pre-push review. The local copy was stale, and nothing asked the remote. The morning's question, whether the public tree is a pure artefact of the private one, was answered for the wrong tree. It is D-76's lesson in another form: ask the thing itself (the index there, the remote here), not a copy of it.

Fixed: - The title edit is restored in the private repo (`d9d68a9`), so the next cut carries it. - `tools/cut_public.ps1` now reads the remote's `main` with `git ls-remote` before deleting anything. It refuses unless the local public copy is at exactly that commit, and says what to do. - The push is leased on the same answer (`--force-with-lease=main:<sha>`), so a change published between the check and the push is not overwritten either.

Tested against a bare scratch remote, all four behaving as intended: - A first cut to an empty remote passed. - A web edit landed on the remote only, and the cut refused, leaving the remote untouched. - With the local copy brought up to date, the cut proceeded. - A second web edit landed before the push, and the leased push was rejected as stale.

Known limit. The guard catches a published change that the local public copy lacks. It cannot tell whether that change has been carried into the private repo. A fetch into the local copy without porting the change satisfies the guard and still loses it. The refusal message says to do both. The reviewer has to check the second.

D-83 — "Classified with your key", to someone who pasted no key (closed 8 Oct)

Found: 8 Oct, checking the P2.9 gate record before committing it. After any classified build, `upload.py:226` shows `CLASSIFIED_NOTE`: "Classified with your key. Anyone the classifier could not place is marked as such rather than guessed at." The choice is `has_key`, which is true for the project key as well as the visitor's. So a free-tier visitor who pasted nothing is told that their key paid for a build the project's key paid for. That is a false claim about whose account was charged. It is the claim D-63 fixed on the failure path, surviving on the success path because the two were worded independently.

Why the gate missed it. Gate check 2 recorded the provenance line, "Classified by claude-sonnet-5, using this demo's shared allowance". That line is correct, and the gate never read the upload panel's own note. Two surfaces answer the same question, and only one was verified. It is the D-18 and D-33 pattern: one figure derived at several sites, with the check pointed at one of them. Searching every on-screen claim about whose key was used found this one site and no other. The other "your key" strings either tell a visitor to add one, or appear only after one has been pasted.

Next step: take the note's wording from the same place as the provenance line. `provenance.py` already holds the three-way answer: "your own API key", "this demo's shared allowance", or nothing claimed when the source is unrecorded, unknown or mixed. And the build records its own source (D-63), so the note should read that, not re-read the key field. Before the step 8 recording: this is the honesty rule, not a rough edge.

Fixed: - `provenance.whose_key(provenance)` is now the one answer to "whose key paid for this build": "your own API key", "this demo's shared allowance", or `None` when the build was keyless, mixed, unrecorded, or recorded outside the vocabulary. `built_line` reads it. Its output is unchanged. - The upload panel's note is now `classified_note(result)`, over the build's own recorded source. It reads "Classified using this demo's shared allowance." or "Classified using your own API key.", and "Classified." when nothing can be claimed. The tail is unchanged.

`tests/test_classified_note.py` (9 tests) covers: - a project-key build through the panel, whose note does not say "your" - a pasted-key build, which says "your own API key" - mixed, unrecorded and pre-D-63 sources, which claim no key - every source, where the note and the provenance line name the same key

Three mutations each fail tests: reverting the wording, deriving it from the key field instead of the build, and dropping the mixed-source rule. 695 tests pass.

D-84 — the About-informed reading is a coin flip (closed 8 Oct)

Found: 8 Oct, by the measurement D-70 carried forward: six runs of the profile comparison on one title and one About text, with identical input each time.

Run	Title only	Title + About
1	15, score 0.95	**abstain**, 0.40
2	15, score 0.95	**abstain**, 0.40
3	15, score 0.95	15, self_enriched 0.95
4	15, score 0.95	15, self_enriched 0.75
5	15, score 0.95	**abstain**, 0.40
6	15, score 0.95	15, self_enriched 0.75

The title-only reading is stable: six identical readings, with the same group and the same score. The About-informed reading is not. It split three abstentions to three assignments, and the three assignments disagree on confidence (0.95, 0.75, 0.75). All the variation is in the reading the panel sets beside the title-only one.

Scope. One title and one About text. This establishes that the path is unstable, not how unstable it is in general.

The title-only side's stability is a result too, and it does not transfer. D-69 measured 15% of titles changing group between runs in the batched tier. On this title, this single-call path did not move at all. The tier, the prompt and the caching all differ, so the two are not comparable, and neither figure carries over to the other.

What the screen does. Since D-70 every branch names no cause and ends with a caveat that each reading "is a single answer". So run 1 reads "…The reading with your description declines to assign a group", followed by the caveat, and run 3 reads "Both readings assign 15…". The causal claim is gone, but on identical input a visitor pressing the button gets one sentence or the other, and nothing on screen says the next press may disagree. The prompt confound D-70 was logged for was systematic but at least repeatable. This is not repeatable.

D-70's Next step anticipated this branch. "If the readings themselves vary, the remedy is repeat calls with a stated agreement." The author's disposition is different: **withdraw the comparison**, because a path this unstable does not support the feature's premise that the two readings can be compared. The comparison stays out of the step 8 recording either way.

The reason, stated plainly. Three of six identical runs declined to assign a group. A panel that sets that reading beside the title-only one invites the visitor to read the difference as their description's doing. Half the time the same description produces the opposite outcome, so it is making a claim about their writing that the code cannot support.

What replaces it. The single About-informed reading, framed honestly as one answer from a classifier that can answer differently when asked again. That caveat is true and sufficient for one reading; it was never sufficient for a comparison. The reading is still one draw, so the caveat is load-bearing, and it is tested as such.

Fixed: - **The panel** (`enrich.py`) makes one call, `classify_from_about`, and shows one reading with `SINGLE_READING_CAVEAT`. Gone: the two columns, `READING_LABELS`, the comparison sentence, and every

"compare the two readings" string (the introduction, the empty-input hint, the button). The introduction now offers "see what the classifier makes of your title and your own words together". The failure handling is kept as it was: a spent day first, then an incomplete answer, the missing-key message, and D-46's clearing of the last result. The ledger charges one call. - `self_enrichment.py.` `describe_comparison`, both caveats, `Comparison.agree` and this module's copy of `_is_confident` are removed. `compare`, `classify_title_only` and `Comparison` remain, documented as command-line only. The CLI prints both raw readings and no product sentence, so D-84's measurement can be run again. - **The session file.** The panel's result is now one `Classification`, so `FORMAT_VERSION` is 2. A version-1 file, which holds two readings, restores its About-informed half (`restore_enrichment_reading`). Read directly, it would have matched no field and rebuilt as an empty reading that looked real. A version-1 build refuses a version-2 file with its existing "newer version" message. - **The design record** (P2.2's definition of done) is amended (`a43e6e4`): what was specified, what the measurement showed, and what ships instead. The "materially improves" criterion is withdrawn with the comparison.

`tests/test_single_reading.py` (5 tests): - through the panel, one call, one reading, the caveat present, and no comparison text - no panel string invites a comparison - an old two-reading file restores its About-informed half - a new file round-trips one reading at version 2 - the CLI prints both raw readings and no comparison sentence

The comparison's wording tests are removed with it. The confidence-gate tests are kept, pointed at the panel's `_is_confident`, because `reading_lines` still separates "names no current occupation" from "could not tell". Five mutations each fail a test. 685 tests pass.

Where this departs from D-70's Next step. D-70 planned repeat calls with a stated agreement for this outcome. The author chose withdrawal: the premise that the two readings can be compared is not supported by a path this unstable, and one honestly caveated reading needs no agreement statistic.

Also seen, not changed. Gate B's table lists "P2.2b — enrichment comparison". Amendment 1 defines P2.2b as *optional per-person notes*, and so does the status document. The P2.2 definition of done never specified a comparison: it specified an improvement, which the side-by-side was built to show.

D-85 — a stale canvas left on the page beneath the current one (closed 9 Oct)

Found: 8 Oct, in the browser verdicts for P2.10 items 1 and 2, on local commit `50af986`, which was never pushed or deployed. The sequence: an upload with no key, giving an unclassified build; then a key pasted, giving a classified network. Beneath the classified dashboard, the page still showed an ungrouped canvas from the earlier, unclassified run. The deployed app (`avian_v1`) does not do this, so it came with `50af986`.

What it is not. One explanation offered was sticky travel: a pinned canvas moving down a stretched column and being repainted against stale content. A sticky element is one element, though, and it always shows the current run's figure. It cannot show the ungrouped canvas of a run that has been replaced. An earlier state on screen means a second element: one from a previous run that the frontend never cleared.

Candidates, both in `50af986`, not distinguished: - The keyed container. `st.container(key="canvas")` gives the block a fixed id (`...-canvas`). Between the unclassified and classified runs, the elements above the canvas change (the offer, the notice, the provenance line), so the block's position moves while its id does not. - The positioning CSS, which stretched the canvas column and made the container sticky.

Disposition. `50af986`'s approach is replaced, not repaired. The chat column is now a panel with its own scroll (`st.container(height=...)`), so the page no longer grows below the canvas, and the canvas needs neither CSS nor a key to stay in place. A test pins that no `position: sticky` returns. **Open until the same sequence is repeated in a browser on the new layout.** If the stale canvas is gone, the cause was one of the two candidates, and which one is not established. If it persists, the cause is something `50af986` did not change, and this entry is wrong about it.

Fixed, with the cause unidentified. Three clean runs of the keyless-then-keyed sequence on the new layout (`a0ac6bf`). Each run was in a fresh private window on a 60-person synthetic export, with the app started without `ANTHROPIC_API_KEY` so that the first build was truly keyless. No leftover canvas appeared

in any of them. Both suspects, the keyed container and the positioning CSS, were removed in one change, so which of them caused it is not known. `tests/test_canvas_layout.py` pins that no `position: sticky` returns.

What the runs do and do not show. The close rests on the mechanism, not on the runs. Both suspects were removed, and nothing positions the canvas any more. The runs used 60 people. The original appeared on a larger, classified network, and that configuration was not reproduced. The review queue is paged at 10, so its height on screen does not grow with the network once ten people need review. A tall column was in any case the condition for the sticky-travel theory this entry rules out, not for a leftover element. The runs corroborate the close; they do not establish it. One run on the 442-person synthetic export is planned as part of the live check after the next cut, which is the configuration that will be recorded.

What a live check can and cannot add. The deployed app classifies on upload when the project key is in Secrets. So the 442-person run in the live check tests the keyed path, and confirms nothing about D-85's sequence. It must not be recorded as "D-85 confirmed on live". The sequence is reachable on live only when the first build is keyless: with the project key absent from Secrets, or on a day whose budget is spent, where the build lands in the fourth state and a pasted key then offers to classify it. Neither is worth arranging before the recording. The close rests on the mechanism and the three local runs.

Consequence for later work. Until a keyed container is shown to be safe on its own, adding one reintroduces an unresolved suspect. That is why `overscroll-behavior: contain`, the fix for P2.10.2's accepted scroll chaining, is deferred: the clean selector for it is a keyed container.

Live check, 10 Oct: no leftover canvas. A 442-person upload to the deployed app (cut `21e5500` in the public repo, as recorded in the P2.9 plan's step 8). Upload, wait for the build, scroll past the attribution line: no leftover canvas. The deploy log identifies the file as the synthetic export: 442 rows read and kept, header at line 3, 12 missing a company. The real export reads 452 rows and drops 10. The container was warm: its first build since the 9 October provision was at 18:29 UTC, with no restart before the live check's builds at 22:08 to 22:09. Those show as three parses 26 and 60 s apart. Why there were three is open (D-90). As the paragraph above requires, this is the keyed path, and it is recorded as a pass of the live check, not as D-85 confirmed on live.

D-86 — dead copy that still describes the withdrawn comparison

Found: 10 Oct, while adding chat history to the session file (P2.10.5). `src/dashboard/session_io.py` defines `STALE_RESULT_NOTE`, "The comparison below was generated on {when} and restored from your file…". D-84 withdrew the comparison, and this string was missed. Nothing references it, so no visitor sees it.

Disposition: left, deliberately, for now. It was found inside a change that can lose data (the session format), and widening that change for copy that is never rendered was the wrong trade. It is logged so it does not vanish with the conversation it was found in. The fix is to delete the constant, or to reword it for a single reading if a restored reading ever gets a note of its own.

D-87 — an intermittent ERROR on a test that cannot error by itself

Found: 10 Oct, during P2.10.5. One full-suite run reported `1 failed, 698 passed, 1 error`. The error was on `tests/test_welcome.py::test_a_non_integer_count_drops_the_size`. (The failure was an expected one, on a test pinning the old format version.) That run took 52 s, against 37 to 42 s for every clean run.

What is known. The test is a pure function test: it calls `demo_caption()` with four values, with no app, no file and no network. A pytest ERROR, rather than FAILED, means a fixture's setup or teardown went wrong. So it came from one of the autouse fixtures that wrap every test: the classification-cache redirect, the spend-ledger redirect (D-81), or D-74's real-API guard. The guard fails a test at teardown if any send was attempted during it.

What is not known. The error text was not captured. The command that ran the suite filtered each result line down to its name. It has not reproduced: 25 runs of `test_welcome.py` alone and nine further full-suite

runs were all clean.

Leading hypothesis, unverified: interference across tests. A thread left running by an earlier `AppTest`-based test, on a run slow enough for one to linger, acts during this test's window. If the actor was the API guard, some code attempted a send, and that would matter. That possibility is why this is logged at Medium rather than dismissed as flakiness.

Next step: capture it. Run the suite with `-rE` (or `-ra`) and keep the full output whenever the result line is not clean. Then the next occurrence names its fixture, and whether it was the API guard.

D-88 — the spike record says the encoder is cached; the code builds it per call

Found: 10 Oct, while timing the build for P2.11. `docs/p2_9b_spike_record.md` says the heavy imports "load once, when the first real build is requested. The loaded model is shared across sessions (`st.cache_resource`), as the spike did." The import half is true: Python keeps torch and sentence-transformers in `sys.modules` after the first build. The model half is not. `src/embeddings._load_model()` constructs `SentenceTransformer(MODEL_NAME)` on every call, with no cache of any kind, and `compute_embeddings` calls it once per build. `cache_resource` has never appeared in `src/` or `app.py`. `git log -S cache_resource` finds it only in the spike record's own commit (`56fcf7a`), where it describes the spike's harness. No later document repeats the claim.

Measured: 10 Oct, three keyless builds of the 442-row export in one process with `NV_TIMING=1` (`0fe5ca2`). The embed stage took 5.75 s, 1.76 s and 1.73 s. Constructing the model alone, after those builds, took 1.32 s. So about three quarters of a warm build's embed stage is rebuilding a model that was already built. One local run each; figures on the deploy will differ. Not measured: memory. From the code, two builds at once each hold their own copy of the model, but what that adds against the 3,072 MB ceiling has not been checked.

The same shape as D-28. A record describes a system nobody checked against the code. D-28's case was stale graph artefacts on disk (`docs/day17_gate_c_addendum.md`). This one is a sentence in a record.

Two halves, both open. - **The record.** A dated note in the spike record saying the model half was never built, with the original sentence left standing. - **The code.** Caching the model is also part of P2.11's answer, not only a documentation fix. 1.3 s is small beside a classifier batch, but it is paid on every upload, including on a warm container, which is the common case. If P2.11 caches the model, the spike record's sentence becomes true, and the note should say from which commit.

D-89 — a refused restore says nothing, and an old success message stays

Found: 10 Oct, at P2.10.5's verdict 3, on the version-2 build (`5b8c3de`, in a worktree). The version-3 file was uploaded to the older build. No "saved by a newer version" message appeared. Beside the refused file, the sidebar still showed "Re-applied 4 of 4 corrections and 1 of 1 note.", left over from a version-2 restore earlier in the same session.

Cause, read from the code. `render_session_io` calls `st.rerun()` straight after `_apply_upload`, whether or not the restore succeeded (`session_io.py:531-532` at HEAD; unchanged since `d5152b0`). Each error path in `_apply_upload` calls `st.error` and returns. The rerun throws away that run's elements, the error with them. On the next run, the guard (`KEY_APPLIED`) stops the file being applied a second time, so the error is never drawn again. `KEY_REPORT` is replaced only on success, so the previous report renders beside the refused file. What the screen then says is the opposite of what happened.

The refusal itself works. The leftover "4 of 4" is the evidence. The version-3 file holds 0 corrections and 0 notes, so a read would have replaced the report with one about those. Nothing was changed.

It affects HEAD, and the live app after the next cut. All three error paths are lost the same way: a file that isn't JSON, a file with no `format_version`, and a file from a newer version.

Seen on HEAD, 10 Oct. In the browser, on the app at HEAD (session code as at `824644f`), `data/demo/provenance.json` was restored. It is 291 bytes, with no `format_version`, and the restore came after a version-2 restore earlier in the same session. No "has no format_version and cannot be read" message

appeared. The earlier "Re-applied 4 of 4 corrections and 1 of 1 note." stayed beside the file. That is the false-success case below, seen on the code the next cut ships. The other two paths, a file that isn't JSON and a newer-version file on this build, are read from the code and have not been seen.

Exposure, and why Medium. It hits any visitor who restores a file the app can't read: one that isn't JSON, one with no `format_version`, or one from a newer version. What they see depends on the session: - **Nothing restored earlier in this session:** silence. The file sits in the uploader, and nothing says it was refused. - **Something restored earlier:** that restore's "Re-applied ..." report stays beside the new file, telling them a restore worked when it did not.

The second case is what makes this Medium, not Low: it puts a false claim on screen, D-63's shape. How often it happens is not known, because the app records no restore attempts. The code has been live since at least 8 September (`d5152b0`).

Why no test caught it. - The unit tests stop before the rerun. They call `_apply_upload` with a fake `st` and read its recorded errors, so the rerun that discards them never runs. - **AppTest can't see it.** It keeps an element from a run abandoned by `st.rerun()` whenever the next run draws nothing in its slot; the browser clears it. A probe on 10 Oct (Streamlit 1.41.1) confirmed this with an error inside a sidebar expander. A repro through the real `render_session_io` under AppTest showed all three errors, so it passes on the broken code. A fix checked only by AppTest would pass with or without the fix: the README's "Tests and verdicts" rule. The test has to assert on state that survives the rerun, such as a stored error, not on AppTest's element tree.

The verdict's input could not have shown a partial read either. The version-3 file carries no corrections, notes or profile text. "Nothing in the session changed" would therefore be true whether the older build refused it or read it partially. Only the message, or the leftover report as above, tells them apart.

Next step, not done: keep the outcome of an upload, error or report, in session state. Clear the previous report when a new file arrives. Render the outcome after the rerun. Not fixed here: it was found inside step 10's verdicts.

Decided, 10 Oct: fixed after the coming cut, as the next item, not someday. The cut doesn't change its exposure, which has been live since September, and the fix stays out of a cut that is already waiting on other checks.

Order changed, 10 Oct (later the same day): D-90 goes first, then this. D-90 was found after the decision above. Its route 1 deletes a visitor's corrections, notes and chat, a data-loss path, and outranks a misleading banner. Both still land before publishing.

D-90 — an upload is built again without being uploaded again

Found: 10 Oct, reading the deploy log after step 3's live check. The log shows three full parses and encodes of the 442-person synthetic export, at 22:08:22, 22:08:48 and 22:09:48 UTC, 26 and 60 s apart. Whether they came from three uploads is not known. Looking in the upload path for how else they could arise found two routes. Both are confirmed with the upload tests' fakes (see "Evidence").

1. **"Use the demo instead" brings the same network back, without the visitor's work.** The button calls `clear_built`, which runs `forget_network_work` (corrections, notes, chat, summaries) and drops the network. It does not empty the uploader (`connections_upload`), and nothing in `upload.py` records that its file was already built. On the rerun, the panel finds no network and a file, so it builds the file again. The visitor asked for the demo and gets their own network back, with their corrections, notes and chat deleted. Nothing on screen says so.
2. **An interaction during a build throws the finished build away.** Any widget interaction while a build runs requests a rerun. The build makes no Streamlit calls, so it runs to the end (D-72). The rerun is raised at the next Streamlit call: the spinner closing. Its `message.empty()` enqueues a message, and every enqueue runs `_maybe_handle_execution_control_request()` (Streamlit 1.41.1, `runtime/scriptrunner/script_runner.py:338-347`). That happens before `set_built`, so the result is lost, and the next run builds the file again.

What a rebuild costs. The session's classification cache survives `forget_network_work`, so a keyed rebuild sends no titles and spends nothing. It does repeat the parse, embedding, UMAP, graph and centrality, about 3 s warm locally. On the project key it also adds one to the daily ledger's network tally, which is counted but never read for a cap. Within one session, runs don't overlap: a requested rerun waits for the running script to stop. So route 2 adds a few seconds after a build, not a second build alongside it.

Evidence, red, reproduced with fakes and not committed (main stays green). Drive `render_upload_panel` with `tests/test_upload.py`'s `FakeStreamlit`, `_Upload(_csv(20))` and a builder that counts its calls: - **Route 1:** a build, then a pass with `click=True` and `on_built` (the button), then a plain pass with the same upload (the rerun). The network is back, and the builder has run twice. - **Route 2:** a pass whose `spinner().__exit__` raises. The build is not kept. A plain pass with the same upload builds again, the second call.

Priority: High. Route 1 deletes what the visitor did, under a button whose label promises something else, and nothing tells them.

Next step, not done: remember the built file by its `file_id`, as `session_io`'s `KEY_APPLIED` does for restores, and never build the same file twice. "Use the demo instead" then leaves the uploader's file ignored, rather than rebuilt. Store the result before anything that can enqueue, so a rerun raised as the spinner closes doesn't lose it. Both routes above become tests.

D-91 — the torch.classes line came back after a cold start

Found: 10 Oct, in the deploy log. P2.9 step 6 set `fileWatcherType = "none"` and recorded that the periodic "Examining the path of torch.classes" lines stopped. It attributed the few that remained to a process without the new config, and said: "If those lines reappear after a cold start on the current config, that explanation is wrong and this needs looking at again." The log shows a fresh provision and clone on 9 Oct at 21:31 UTC, followed by the line at 18:59 and 19:28 UTC on 10 Oct. The first build in that container was at 18:29, so the lines came about 30 and 60 minutes after it.

This assumes the deployed cut carries `.streamlit/config.toml`, as step 6 says it does. That has not been checked in the public repo.

Effect: log noise only. Nothing a visitor sees, and no behaviour change.

Next step, not done: confirm `config.toml` is in the public repo's tree. If it is, find what else in Streamlit 1.41 walks module paths. The 30-minute spacing after the first build is a lead, not a finding.

Closed after the batch (22 September 2026)

ID	Defect	Disposition	Commit
D-47	Phase 1.5 evaluation may carry padded abstentions	Remeasured offline: 0 of 100 titles changed; config E unchanged; decision stands (see above)	`git log --grep D-47`
D-49	Dead HDBSCAN constants; stale `config.py` docstring	Removed, after confirming nothing read them	`f023184`
D-50	Correction-miss restore path never executed	Exercised manually; passed (see above)	none needed
D-52	The deployed environment ran pyarrow 24.0.0, not the locked 25.0.0 (found by the P2.9b spike)	`pyarrow==24.0.0` pinned; locks recompiled, only pyarrow moved	`git log --grep D-52`

ID	Defect	Disposition	Commit
D-51	A note was kept only on Ctrl+Enter or leaving the box, with no visible save control (found during D-50)	*Save note* button in a form; typing alone no longer saves, and the help line says so; 4 tests	`git log --grep D-51`

Closed in the documentation batch (21–22 September 2026)

ID	Defect	Disposition	Commit
D-21	The standing-commands table lacked `python -m tools.<name>` and listed `git add .`	Standing commands moved to a *Development* section in `README.md`; dated note on the old table	`2e4b903`
D-22	Three comments said `apply_corrections` has 26 tests; it has 9	Comments point at `tests/test_state.py` instead of counting	`0aa6a89`
D-29	UMAP parameters and seed duplicated across the 2-D and 3-D modules, with a rationale the tracked evidence no longer supported	One copy in `config.py` with the real rationale; 3 tests	`6557160`
D-30	matplotlib used but missing from `requirements.txt`	In the dev lock	`1041cb6`
D-31	Build and runtime dependencies not split	`import_closure` tool (`d6bb74a`); `graph.py` imports matplotlib lazily, with a runtime-boundary test (`57e4622`); runtime and dev locks with `verify_manifests` (`1041cb6`)	as listed
D-39	Two copies of the exemption-marker parser; the two application rules undocumented	Shared `tools/_markers.py` stating both rules; 5 tests; no change in findings	`ec52a98`
D-40	`diff_rebuild` and `audit_title_churn` hard-coded a gitignored snapshot	Required `--old` argument, plain error on a missing file; 7 tests; output identical	`23725ee`
D-42	Failed classifier batches cached as abstentions	Fixed on Day 19 (`git log --grep D-42`); failure-mode record (`4606629`); dated correction notes on eight records (`6e10000`)	as listed

ID	Defect	Disposition	Commit
D-43	Dead Day 1 scaffold `app/streamlit_app.py`, a trap at deploy	Deleted	`393b134`
D-44	`tools/probe_unparsed.py` read the node table directly	Reads through `load_base_frame()`	`d35e638`
D-45	Summaries at a 400-token budget, `stop_reason` never read	Budget 8000; a cut-off reply is never shown or cached; harness fails instead of skipping; probe committed as evidence	`c60b51e`
D-46	A failed self-enrichment call reported to the user as an abstention; a stale result left under the error	Raises `ClassificationFailed`; the panel clears the old result and says the answer was incomplete; 7 tests	`47dd5d0`

Also closed, no commit needed: the 441/442 record. When a connection was removed and restored before Gate C, the restoration was documented in that commit's message and the rebuild was verified byte-identical at 442 rows. It was listed in `docs/phase2_scope_revision.md` from a stale queue.

Carried into later work

These are not defects in current behaviour; they are inputs a later item must handle.

For	Item
P2.9b	`graph.load_nodes()` reads `data/labels/cluster_labels.csv`, which is gitignored, so a fresh deploy will not have it.
P2.9b	The embeddings cache keys on row count alone, so a rebuild at the same row count reuses stale vectors.
P2.9b	Partial-failure semantics must be explicit: when a build loses a batch, it aborts or returns a network that states what is missing.
P2.9	`README.md` still says roughly 40% of a network needs review; the corrected figure is 29.0%. The README is replaced at P2.9.
P2.9	Set Community Cloud's Python version to 3.11 under Advanced settings, to match the lock.
D-70	~~Measure the profile comparison's own spread before choosing between prompt work and repeat calls.~~ Measured 8 Oct: see D-84.

Closed during P2.9b (23 September 2026)

ID	Defect	Disposition	Commit
D-53	Two `coverage_counts` implementations over the same three states: `assistant.py` and `summaries.py`. Gate D recorded them as one source	`assistant.coverage_sentence` reads `summaries.coverage_counts`; duplicate deleted; identity test, plus a raise on a frame with no `display_state`	`efa475f`

Closed during P2.9b (26 September 2026)

ID	Defect	Disposition	Commit
D-57	`key_source()` changed from returning `None` for "no key" to returning the sentinel `"none"`; every truthiness check on it silently inverted	`upload.py` uses `key_source(store) != KEY_NONE`; `key_source` documents that its return is always truthy	`git log --grep D-57`

Closed during P2.9b (28 September 2026)

ID	Defect	Disposition	Commit
D-62	Tests reading `ANTHROPIC_API_KEY` from the ambient shell pass or fail depending on the developer's environment, not the suite	Swept: full suite run with the key unset and with a fake value, 537 passed both times; the two known cases fixed with `monkeypatch.delenv` (see above)	none needed
D-63	`build_network` defaulted `source` to `KEY_PROJECT` and the upload panel never passed it, so a failure on the visitor's own key was explained as ours	`source` defaults to `None`; the panel passes `key_source(store)`; the notice and card take the build's `source`, with a neutral phrase for `None`; regression guard in `tests/test_app.py`. D-62's recipe re-run: 566 passed with the key unset and with a fake value	`git log --grep D-63`
D-64	A test that classifies without an injected cache path writes to the real classification cache, and nothing fails	`CACHE_PATH` resolved at call time; autouse `conftest.py` fixture redirects it per test; a session backstop fails the run if the real file's hash changes; each defaulting path tested	`git log --grep D-64`

Closed during P2.9b (29 September 2026)

ID	Defect	Disposition	Commit
D-58	`welcome.py` hardcoded the demo's 442-person count	Read from `data/demo/provenance.json` via `load_demo_provenance()`; size omitted when unreadable; `test_demo.py` pins the recorded count to the table's rows	`git log --grep D-58`
D-65	Corrections and notes, keyed by row position, survived a change of network; the demo's corrections appeared on a visitor's own people as theirs	`forget_network_work()`, called by `clear_built()`, which the upload panel now calls before `set_built()`; retry keeps the work	`git log --grep D-65`
D-66	The key field's expander nested inside the sidebar's "Your network" expander; Streamlit refused it and the dashboard raised on every run past the welcome screen	`boxed=False` from the sidebar renders the field under a caption; AppTest smoke test boots `app.py` for welcome, demo and a built network	`git log --grep D-66`
D-67	`st.rerun()` passed as `on_key` and called from inside the key field's `on_change` callback; Streamlit rendered a no-op warning on every valid paste	Neither call site passes `on_key`; docstring states the rule; AppTest paste runs on the welcome screen and sidebar assert no warning	`git log --grep D-67`
D-68	Reported: the project key would not reach the app on Community Cloud, which has no shell	Not a defect as reported: `streamlit run` copies root-level secrets into `os.environ` at start. The `st.secrets` read was not added (it renders an error on keyless runs). Test pins root-level found and nested not; README deploy note; `.streamlit/secrets.toml` gitignored	`git log --grep D-68`

ID	Defect	Disposition	Commit
D-70	The profile comparison's wording attributed differences to the user's description, though the two readings use different prompts and are one call each	Every branch reports, names no cause, and ends with a caveat naming both; module docstring retreats to "a place to see what changes, not a test of it"; the spread measurement is carried	`git log --grep D-70`

Closed during P2.9 (6 October 2026)

ID	Defect	Disposition	Commit
D-71	`build_network` and `retry_unclassified` defaulted `cache_path` to the shared, tracked classification cache, so a session build wrote real job titles to disk	A dict is accepted as the cache; both functions default to a private mapping; one mapping per session, passed by the upload panel and the retry handler; `build_demo.py` names the shared file; 11 tests, D-64's isolation test reversed	`git log --grep D-71`
D-74	Four tests made real Anthropic calls whenever `ANTHROPIC_API_KEY` was set in the shell, and passed either way	Autouse guard at the SDK's send, raising a `BaseException` and failing at teardown on any attempt; the four pin the classification axis (two keyless, one keyed, one both); guard test through the chat's broad handler	`git log --grep D-74`
D-75	The README said deleting `data/cache/` is always safe; four files in it are tracked, one of them the demo's classification cache	Caches section names the four and points at `git checkout -- data/cache`; logged after the fix	`197102b`
D-76	Tracked files under `data/` derived from the real network, added before the ignore rules and so invisible to every `.gitignore` review; one, a PNG, renders real job titles	PNG untracked and ignored by name; four aggregate files (graph metrics, three Phase 1 sweeps) kept; found by `git ls-files data/`. Amended 6 Oct: graph metrics then untracked too, as no longer evidence for anything asserted (dated note)	`git log --grep D-76`

ID	Defect	Disposition	Commit
D-78	`docs/project_state_decisions_log.md` named five real connections, in prose, outside every directory a real-data check had looked in	Redacted in place in the `.md` and `.pdf`, with a dated note naming no one; found by a full-name scan of all history, then a surname-only pass at HEAD; limits recorded	`git log --grep D-78`
D-77	Two Phase 1 methodology records (summary, maths) presented the 83-profile corroboration and HDBSCAN as the method, with no note that Phase 1.5 overturned it on real data	Dated notes at the top of each, giving the real-network figures and pointing to `phase1_5_methodology.md`; aimed at §10, §11 and §13 of the summary and at the maths document's "real values" table	`git log --grep D-77`
D-79	`ingestion.load_profiles()` still passed `path=` after `719d64f` renamed the parameter, so every command-line module raised `TypeError` for twelve days; no test called it	Passed positionally; `tests/test_ingestion.py` calls it on the synthetic export (fails against the old call); the chain reruns end to end with real data, with no second stale call	`git log --grep D-79`
D-81	`Meters`, `DailyLedger` and `GenerationMeter` were never constructed by the app: builds and retries on the project key ran with no generation cap and no daily ledger	`metering.meters_for` on the project key, passed by the upload panel and the retry handler; `retry_unclassified` takes `meters`; network tally recorded; 7 tests through the panels; conftest ledger guard; ephemeral-disk limit recorded	`git log --grep D-81`
D-80	The ten-question chat allowance was built and tested but never wired in; chat, summaries and enrichment charged nothing to the daily ledger	Chat panel applies and shows the allowance, counting answered questions only; `LedgerClient` charges all three to the ledger, never the question count; spent-day refusals point at the key field; 9 tests through the panels; an at-cap boundary bug found and fixed	`git log --grep D-80`

ID	Defect	Disposition	Commit
D-82	The first scripted cut force-pushed over a title edit made in GitHub's web editor on the public repo, because every check compared against a stale local copy	Edit restored in the private repo; the script asks the remote for its `main`, refuses unless the local copy matches, and leases the push on it; tested against a scratch remote	`git log --grep D-82`
D-83	The upload panel's note said "Classified with your key" after any classified build, including one the project's key paid for	`provenance.whose_key` is the one answer to whose key paid; the note and the provenance line both read it, over the build's recorded source; 9 tests	`git log --grep D-83`
D-84	The profile comparison set an About-informed reading that abstained in three of six identical runs beside a title-only reading that never moved	Comparison withdrawn: the panel shows one reading with a one-answer caveat; CLI keeps both readings as the measurement path; session format 2, with version-1 files restoring their About-informed half; P2.2 amended; 5 tests	`git log --grep D-84`
D-85	With the canvas pinned by sticky CSS in a keyed container (local `50af986`, never deployed), a stale ungrouped canvas from an earlier run stayed on the page beneath a classified one	Approach replaced by an own-scroll chat panel with no CSS and no key (`a0ac6bf`); three clean runs of the sequence; cause between the two suspects unidentified	`git log --grep D-85`